An **addressing mode** is the mechanism a processor uses to figure out *where* the actual operand (data) is located for an instruction — is it embedded right in the instruction, sitting in a register, sitting in memory at a fixed address, or somewhere memory that needs an extra calculation to find?

Key points from your slide:

- **Purpose**: Addressing modes specify *how* to locate the operand data that an instruction needs to work with.
- **Complexity varies by ISA**:
  - Complex ISAs (like x86) support many addressing modes.
  - **Load-store architectures** (like RISC processors — ARM, MIPS, RISC-V) are simpler and support only a limited set of addressing modes, since only load/store instructions touch memory — everything else operates on registers.
- **Common addressing modes** (you'll cover these in detail later in the course):
  - **Immediate** – the operand value is part of the instruction itself (e.g. `ADD R1, #5`)
  - **Direct** – the instruction gives the actual memory address of the operand
  - **Indirect** – the instruction gives an address that *points to* another address, which holds the operand
  - **Register** – the operand is in a register
  - **Register Indirect** – a register holds the address of the operand (not the operand itself)
  - **Indexed** – address = base address + value in an index register (useful for arrays)
  - **Stack** – operand is implicitly on the stack (push/pop style)
  - **Relative** – address = current instruction address (PC) + offset
  - **Autoincrement / Autodecrement** – like register indirect, but the register is automatically incremented/decremented after (or before) use — handy for stepping through arrays
  - **Based** – similar to indexed, address = base register + offset
- Not every processor implements all of these — it depends on the ISA design.

Think of it like giving someone directions to find an item: you could tell them the item itself (immediate), give them the exact shelf location (direct), give them a note that says "the location is written on this other paper" (indirect), or tell them "look in your pocket" (register) — different levels of indirection for finding the same thing.

Immediate Addressing

The core idea: **the operand isn't stored somewhere else and fetched — it's baked directly into the instruction itself.**

Why it matters

- **No memory reference needed**: With most addressing modes, the CPU has to go fetch the operand from a register or memory location. Here, once the instruction is decoded, the data is already sitting right there in the instruction word. That's it — no extra lookup step.
- **This makes it fast**: Since there's no extra memory access, immediate addressing has the lowest latency of any addressing mode.
- **But it has a limited range**: The instruction has a fixed width (say 32 bits), and part of that has to be used for the opcode. Whatever bits remain for the "immediate data" field limits how large a value

you can represent. You can't stuff a huge 64-bit number into a small leftover field — so this mode is only good for small constants.

The instruction layout (from the diagram)

```
| opcode | immediate data |
```

The instruction is literally split into two parts: the operation to perform, and the actual value to use as an operand. Compare this to direct/indirect addressing, where that second field would instead hold an *address* pointing to where the data lives — here it holds the *data itself*.

Walking through the examples

`ADDI #25 // ACC = ACC + 25`

- `ADDI` = "add immediate" opcode
- `#25` = the immediate value itself (the `#` symbol is a common convention meaning "this is a literal value, not an address")
- Effect: take whatever is in the accumulator (ACC), add the literal number 25 to it, store the result back in ACC
- No register or memory needs to be read to get "25" — it's already part of the instruction

`ADDI R1, R2, 42 // R1 = R2 + 42`

- Three-operand form: two registers plus one immediate value
- `R2` is read from a register (register addressing for that operand)
- `42` is immediate — embedded directly in the instruction
- Result is written into `R1`
- This shows immediate addressing is often *combined* with other addressing modes in the same instruction — not every operand has to use the same mode

Quick mental model

Think of a recipe card: instead of writing "use the amount of sugar written on the sugar jar" (indirect/memory reference), immediate addressing is like writing "add 2 tablespoons of sugar" directly on the card. You don't need to go check anywhere else — the value's right there.

Here are 10 questions covering immediate addressing:

1. **Define immediate addressing mode.**
2. In immediate addressing, does the CPU need to make a memory reference to access the operand? Explain why or why not.
3. Why is immediate addressing considered **fast** compared to other addressing modes?
4. Immediate addressing has a **limited range** for the values it can represent. Explain why.
5. Draw and label the instruction format used in immediate addressing.
6. In the instruction `ADDI #25`, identify which part is the opcode and which part is the immediate data. What does this instruction do?

7. What does the `ACC` mean in `ADDI #25 // ACC = ACC + 25`? Rewrite the effect of this instruction in your own words.
8. Consider `ADDI R1, R2, 42 // R1 = R2 + 42`. Which operand(s) use immediate addressing, and which use another addressing mode? Explain.
9. If an instruction word is 32 bits wide and the opcode takes up 8 bits, how many bits are left for the immediate data field, and what does this imply about the maximum value that can be represented?
10. Give one real-world advantage and one real-world limitation of using immediate addressing in program design (e.g., loop counters, constants).

1. Define immediate addressing mode.
Immediate addressing is an addressing mode where the operand value is embedded directly within the instruction itself, rather than being stored in a register or memory location that needs to be fetched separately.

2. Does the CPU need a memory reference to access the operand?
No. Since the operand is part of the instruction word itself, once the instruction is fetched and decoded, the operand is already available — no additional memory access is needed to retrieve it.

3. Why is immediate addressing fast?
Because there's no extra step to fetch the operand from memory or a register. Other addressing modes may require one or more memory accesses to locate the actual data, but immediate addressing skips this entirely, reducing execution time.

4. Why is the range limited?
Because instructions have a fixed bit-width (e.g., 32 bits), and part of that width must be reserved for the opcode. Whatever bits remain for the immediate data field determine the maximum size of value that can be represented — so only relatively small constants can be encoded this way.

5. Instruction format:

```
| opcode | immediate data |
```

The opcode specifies the operation to perform; the immediate data field holds the actual operand value used in that operation.

6. `ADDI #25` — identify opcode and immediate data:

- Opcode: `ADDI` (add immediate)
- Immediate data: `25`
- Effect: Adds the literal value 25 to the current value in the accumulator (ACC), then stores the result back in ACC.

7. What does ACC mean, and rewrite the effect:
ACC refers to the **accumulator**, a special register commonly used to hold the result of arithmetic operations. In plain terms: "Take the current value stored in the accumulator, add 25 to it, and save the new result back into the accumulator."

8. `ADDI R1, R2, 42 // R1 = R2 + 42` — which operands use which mode:

- `R2` uses **register addressing** — its value is read from register R2.

- `42` uses **immediate addressing** — it's a literal constant embedded in the instruction.
- `R1` is the **destination register** where the result is written.
  This shows a single instruction can combine multiple addressing modes for different operands.

9. 32-bit instruction, 8-bit opcode — bits left for immediate data:
32 − 8 = **24 bits** remain for the immediate data field.
This means the maximum unsigned value representable is $2^{24} - 1 = 16{,}777{,}215$ (or a smaller signed range if two's complement is used, e.g., roughly −8,388,608 to 8,388,607).

10. One advantage and one limitation in real-world program design:

- **Advantage:** Efficient for operations involving small, known constants — e.g., initializing a loop counter (`i = 0`) or incrementing by a fixed step (`i = i + 1`), since these values are known at compile time and don't need memory storage.
- **Limitation:** Cannot be used for large constants or values that aren't known until runtime (e.g., user input or data computed dynamically), since the instruction's fixed-width immediate field can't hold arbitrarily large or variable data.

Here are more practice questions in that style — writing/interpreting instructions and identifying addressing modes:

1. Write an immediate addressing instruction that subtracts 15 from the accumulator. Label the opcode and the immediate value.

2. For the instruction `MOVI R3, #100`, identify the opcode, the destination, and the immediate value. Explain what the instruction does.

3. Given `MULI R1, R2, 7 // R1 = R2 × 7`, identify which operand uses immediate addressing and which uses register addressing. Justify your answer.

4. Draw the instruction format (as opcode/field boxes) for a two-operand immediate instruction like `ADDI R1, 50`, where R1 is a register and 50 is an immediate value.

5. Write an instruction using immediate addressing that loads the value 200 into register R5. Then draw its instruction format showing the opcode and immediate data fields.

6. Given the instruction `CMPI R4, #10 // compares R4 with 10`, is this immediate addressing, register addressing, or a combination of both? Explain your reasoning.

7. A 16-bit instruction reserves 6 bits for the opcode. Draw the instruction format and calculate how many bits remain for the immediate data field.

8. Write two different instructions — one using purely immediate addressing (single operand) and one combining immediate addressing with register addressing (like the `ADDI R1, R2, 42` example). Label each operand's addressing mode.

9. Given the instruction `SUBI R2, R2, 1 // R2 = R2 - 1`, explain in words what this instruction does and why it's a common pattern in loops (hint: think of a loop counter decrementing).

10. Draw the general instruction format for immediate addressing (opcode | immediate data), then modify it to show a three-operand version (opcode | destination register | source register | immediate data) similar to `ADDI R1, R2, 42`.

Here are the model answers with diagrams:

1. Subtract 15 from the accumulator (immediate addressing)

```
SUBI #15   // ACC = ACC - 15
| opcode: SUBI | immediate data: 15 |
```

- Opcode: `SUBI` (subtract immediate)
- Immediate value: `15`
- Effect: Takes the current value in ACC, subtracts 15, stores result back in ACC.

2. `MOVI R3, #100`

- Opcode: `MOVI` (move immediate)
- Destination: `R3`
- Immediate value: `100`
- Effect: Loads the literal value 100 directly into register R3. No memory or other register is read — the value comes straight from the instruction.

3. `MULI R1, R2, 7 // R1 = R2 × 7`

- `R2` → **register addressing** (its value is fetched from the register)
- `7` → **immediate addressing** (a literal constant embedded in the instruction)
- `R1` → destination register that receives the result
- Justification: R2 requires a register read to get its value, while 7 needs no fetch at all — it's already part of the instruction word.

4. `ADDI R1, 50` — instruction format

```
| opcode: ADDI | R1 (register) | immediate data: 50 |
```

- First field: operation to perform
- Second field: destination register (R1)
- Third field: literal value (50) added to R1

5. Load 200 into R5 using immediate addressing

```
MOVI R5, #200   // R5 = 200
| opcode: MOVI | R5 (register) | immediate data: 200 |
```

6. `CMPI R4, #10 // compares R4 with 10`

- This is a **combination** of addressing modes:

    - `R4` → register addressing (its stored value is read for comparison)
    - `#10` → immediate addressing (literal constant, no fetch needed)
- The instruction compares the value in R4 against the immediate constant 10 — commonly used to set condition flags for a branch.

7. 16-bit instruction, 6-bit opcode — instruction format

```
| opcode (6 bits) | immediate data (10 bits) |
```

16 − 6 = **10 bits** remain for immediate data.
Max unsigned value = $2^{10} - 1$ = **1023**.

8. Two instructions — pure immediate vs. combined

Pure immediate (single operand):

```
ADDI #30   // ACC = ACC + 30
| opcode: ADDI | immediate data: 30 |
```

- `30` → immediate addressing only.

Combined (register + immediate):

```
ADDI R1, R2, 42   // R1 = R2 + 42
| opcode: ADDI | R1 (dest, register) | R2 (source, register) | immediate data: 42 |
```

- `R2` → register addressing
- `42` → immediate addressing
- `R1` → destination register

9. `SUBI R2, R2, 1 // R2 = R2 - 1`

- Effect: Reads the current value of R2 (register addressing), subtracts the literal constant 1 (immediate addressing), and writes the result back into R2.
- Why common in loops: This is the classic pattern for **decrementing a loop counter** (e.g., `for i = 10 downto 0`). Since the decrement amount (1) is fixed and known at compile time, immediate addressing is the natural and most efficient choice — no need to store "1" anywhere else.

10. General format → three-operand version

Two-field basic format:

```
| opcode | immediate data |
```

Three-operand version (like `ADDI R1, R2, 42`):

`| opcode | R1 (destination register) | R2 (source register) | immediate data: 42 |`

- Opcode: operation to perform (e.g., `ADDI`)
- R1: register that receives the result
- R2: register providing one operand (read via register addressing)
- Immediate data: the constant value (42) provided directly in the instruction

Direct Addressing

The core idea: **the instruction doesn't hold the operand itself — it holds the memory address where the operand lives.**

How it differs from immediate addressing

- In **immediate addressing**, the instruction field contains the *actual data*.
- In **direct addressing**, the instruction field contains an *address*, and the CPU must go to memory, look up that address, and fetch the value stored there.

That's why direct addressing needs a memory access, while immediate addressing doesn't.

Key characteristics from the slide

- **Instruction contains a field with the memory address** of the operand — not the data itself.
- **Single memory access is required**: the CPU reads the address from the instruction, then makes exactly *one* trip to memory to fetch the actual operand. (Compare this to indirect addressing later, which needs *two* memory accesses — one to get another address, one to get the actual data.)
- **No additional calculations needed**: the address given in the instruction is used exactly as-is — no adding offsets, no combining with register values. This makes it simpler (and faster) than modes like indexed or based addressing.
- **Limited address space**: since the instruction has a fixed number of bits available for the address field (e.g., 16 bits), you can only directly address up to 2^{16} memory locations. If the memory is larger than that, direct addressing simply can't reach the rest of it.

Walking through the example

```
ADD R1, 20A6H // R1 = R1 + Mem[20A6]
```

- `ADD` = opcode
- `R1` = destination register (also one source operand, via register addressing)
- `20A6H` = a memory *address* (in hexadecimal), not a value
- Execution: the CPU takes the value currently in R1, then goes to memory location `20A6H`, fetches whatever value is stored there (`Mem[20A6]`), adds it to R1, and stores the result back in R1.

So this instruction actually combines **two addressing modes**: R1 uses register addressing, while `20A6H` uses direct addressing.

Mental model

Think of direct addressing like being handed a piece of paper that says "the item you need is in **locker #206**." You don't get the item directly (that's immediate addressing), and you don't get a note that says "go to locker #100, and *that* locker has another note telling you where to really go" (that's indirect addressing) — you go straight to locker 206, open it once, and there's your item.

Here are practice questions on direct addressing, in the writing-instructions and drawing-boxes style:

1. Write a direct addressing instruction that adds the value stored at memory address `30B4H` to register R2. Label the opcode, register, and address.

2. For the instruction `SUB R3, 1F2AH // R3 = R3 - Mem[1F2A]`, identify which parts use register addressing and which parts use direct addressing.

3. Draw the instruction format (as opcode/field boxes) for `ADD R1, 20A6H`, showing the opcode field, register field, and memory address field.

4. Write a direct addressing instruction that loads (moves) the value stored at memory address `4000H` into register R5.

5. Given the instruction `MOV R1, 3000H // R1 = Mem[3000]`, explain step-by-step what the CPU does to execute this instruction, and state how many memory accesses are required.

6. A CPU has an instruction format with 8 bits for the opcode and 16 bits for the direct address field. Draw the instruction format as boxes, and calculate the maximum number of memory locations that can be directly addressed.

7. Write two instructions — one using immediate addressing and one using direct addressing — that both appear to "add 42" to R1, but explain why their instruction fields and execution differ.

8. Given `CMP R2, 5050H // compares R2 with Mem[5050]`, is this pure direct addressing, pure register addressing, or a combination? Justify your answer.

9. Draw the instruction format for a direct-addressing instruction with a destination register, a source register, and a direct memory address — e.g., `ADD R1, R2, 20A6H // R1 = R2 + Mem[20A6]`.

10. Explain why direct addressing needs only **one** memory access, while indirect addressing (which you may cover next) needs **two**. Use a diagram or box representation to illustrate the difference.

1. Add value at memory address `30B4H` to R2

```
ADD R2, 30B4H   // R2 = R2 + Mem[30B4]
| opcode: ADD | register: R2 | address: 30B4H |
```

- Opcode: `ADD`
- Register: `R2` (register addressing — holds one operand and receives the result)
- `30B4H`: memory address (direct addressing — CPU fetches the value stored there)

2. `SUB R3, 1F2AH // R3 = R3 - Mem[1F2A]`

- `R3` → **register addressing** (its current value is read directly from the register, and the result is written back to it)
- `1F2AH` → **direct addressing** (this is a memory address; the CPU accesses memory once to fetch the value stored there)

3. Instruction format for `ADD R1, 20A6H`

```
| opcode: ADD | register: R1 | address: 20A6H |
```

- Opcode field: specifies the operation
- Register field: specifies R1 (source + destination)
- Address field: specifies the memory location to fetch the second operand from

4. Load value at address `4000H` into R5

```
MOV R5, 4000H   // R5 = Mem[4000]
| opcode: MOV | register: R5 | address: 4000H |
```

5. `MOV R1, 3000H // R1 = Mem[3000]` — step by step

1. CPU fetches and decodes the instruction.
2. CPU reads the address field: `3000H`.
3. CPU accesses memory location `3000H` and retrieves the value stored there.
4. CPU loads that value into register R1.

Memory accesses required: 1 (this is the defining feature of direct addressing — a single trip to memory).

6. 8-bit opcode + 16-bit direct address field

```
| opcode (8 bits) | address (16 bits) |
```

Maximum addressable memory locations = 2^{16} = **65,536 locations**.

7. Immediate vs. direct — "add 42" to R1

Immediate addressing:

```
ADDI R1, #42   // R1 = R1 + 42
| opcode: ADDI | register: R1 | immediate data: 42 |
```

- The value `42` is used directly — no memory access.

Direct addressing:

```
ADD R1, 42H   // R1 = R1 + Mem[42]
| opcode: ADD | register: R1 | address: 42H |
```

- Here `42H` is a memory *address* (location 42 in hex), not the value 42. The CPU must fetch whatever value is stored at that memory location — this requires one memory access, unlike the immediate version.

(This highlights a common point of confusion: a number appearing in an instruction could either be a literal value or an address, depending on the addressing mode used.)

8. `CMP R2, 5050H // compares R2 with Mem[5050]`

This is a **combination** of addressing modes:

- `R2` → register addressing (value read directly from the register)
- `5050H` → direct addressing (CPU accesses memory location 5050H to get the value to compare against)

9. `ADD R1, R2, 20A6H // R1 = R2 + Mem[20A6]`

```
| opcode: ADD | R1 (destination register) | R2 (source register) | address: 20A6H |
```

- R1: destination (register addressing)
- R2: source operand (register addressing)
- `20A6H`: memory address of the second operand (direct addressing) — one memory access needed to fetch `Mem[20A6]`

10. Why direct addressing needs 1 memory access vs. indirect needing 2

Direct addressing:

```
Instruction → [ address A ] → Memory[A] = operand
```

- Step 1: Read address A from the instruction.
- Step 2: Access Memory[A] → this IS the operand. **(1 access)**

Indirect addressing:

```
Instruction → [ address A ] → Memory[A] = address B → Memory[B] = operand
```

- Step 1: Read address A from the instruction.
- Step 2: Access Memory[A] → this gives you another address, B (not the data yet).
- Step 3: Access Memory[B] → this IS the operand. **(2 accesses)**

In short: direct addressing points straight at the data, while indirect addressing points at a *pointer* to the data — requiring an extra hop through memory.

Immediate vs. Direct Addressing

Aspect	Immediate Addressing	Direct Addressing

What the instruction field holds	The actual operand **value**	The **memory address** of the operand
Memory access needed	**None** — the value is already in the instruction	**One** memory access to fetch the value at the given address
Speed	Fastest — no extra fetch step	Slower than immediate (extra memory access adds latency)
Flexibility of the operand	Fixed at compile time — operand is a constant embedded in the code	Can point to a value in memory that may change during program execution
Range/size limitation	Limited by the number of bits left after the opcode in the immediate field — restricts how large a constant can be	Limited by the number of bits in the address field — restricts how much memory can be directly addressed
Typical use case	Loop counters, fixed constants, small known values (e.g., `ADDI #1`)	Accessing variables stored in memory whose values can change (e.g., `ADD R1, 20A6H`)
Example	`ADDI #25 // ACC = ACC + 25`	`ADD R1, 20A6H // R1 = R1 + Mem[20A6]`
Instruction format	`| opcode | immediate data |`	`| opcode | register | address |`

The key distinction in one sentence:

Immediate addressing gives you the data itself; direct addressing gives you a map to where the data is stored.

Why this distinction matters

- Immediate addressing is ideal when the value is *known in advance and never changes* — like adding a fixed constant.
- Direct addressing is necessary when the operand is a *variable* stored in memory — something that could hold different values at different points during program execution, so you can't hardcode it into the instruction.

Quick way to remember

Think of immediate addressing as writing the number directly on a form ("Enter 25 here"), while direct addressing is like writing "see box 20A6 for the number" — you still have to go open that box to find out what's inside.

Indirect Addressing

The core idea: **the instruction doesn't hold the operand, and it doesn't even hold the operand's address directly — it holds the address of *another location* that itself contains the operand's address.** There's an extra layer of pointing.

How it builds on direct addressing

- **Direct addressing**: instruction → address → **operand**
- **Indirect addressing**: instruction → address → **another address** → operand

So indirect addressing adds one extra "hop" compared to direct addressing.

Key characteristics from the slide

- **The instruction field holds a memory address**, but that address doesn't point straight to the data — it points to a location that *itself contains another address*, and that second address finally points to the actual operand.
- **Two memory accesses are required**:
  - First access: go to the address given in the instruction → get back another address (a pointer).
  - Second access: go to *that* address → get the actual operand value.
- **Slower than direct addressing** — because of that extra memory access, it takes more time to actually get the operand.
- **But it can access a much larger address space**: since the instruction field only needs to hold the address of a *pointer* (not the operand's address directly), that pointer itself — stored in memory — can be as wide as needed. It isn't constrained by however many bits the instruction format allocates. This is the key advantage over direct addressing, which is capped by the size of its address field.

Walking through the example

```
ADD R1, (20A6H) // R1 = R1 + (Mem[20A6])
```

Note the parentheses around `20A6H` — this is the typical notation that signals *indirection*.

- `ADD` = opcode
- `R1` = register (register addressing — holds one operand, receives the result)
- `(20A6H)` = indirect address

Execution:

1. CPU reads `20A6H` from the instruction.
2. CPU accesses memory location `20A6H` — but what's stored there is *not* the operand. It's another address, say `4F10H`.
3. CPU then accesses memory location `4F10H` — **this** is where the actual operand value is stored.
4. That value is added to R1, and the result is stored back in R1.

Mental model

Think of it like a treasure hunt with two clues:

- **Direct addressing** = a note that says "the treasure is in locker #206."
- **Indirect addressing** = a note that says "go to locker #100 — inside is *another* note telling you which locker has the real treasure." You open locker #100, find a note saying "locker #350," then go open locker #350 to finally get the treasure.

That extra step is exactly why indirect addressing needs two memory accesses instead of one — but it also means the "real" locker number written inside locker #100 can be any size, since it's stored in memory rather than squeezed into the treasure hunt clue itself.

Box Diagram — Indirect Addressing

Instruction format:

```
| opcode: ADD | register: R1 | address: 20A6H |
```

Execution flow (the two-hop memory access):

```
Instruction
+-------+------+----------+
|  ADD  |  R1  |  20A6H   |
+-------+------+----------+
                    |
                    | (1st memory access)
                    v
              Memory[20A6H]
              +----------+
              |  4F10H   |   <-- this is NOT the operand, it's another address
              +----------+
                    |
                    | (2nd memory access)
                    v
              Memory[4F10H]
              +----------+
              |   35     |   <-- THIS is the actual operand
              +----------+

Result: R1 = R1 + 35
```

Compact box comparison:

```
Direct:    [ instruction: address A ] --> [ Mem[A] = operand ]
(1 access)

Indirect:  [ instruction: address A ] --> [ Mem[A] = address B ] --> [ Mem[B] =
operand ]   (2 accesses)
```

Exercises — Indirect Addressing

1. Write an indirect addressing instruction that subtracts the operand pointed to by address `3B20H` from register R3. Use correct notation, then draw its instruction format as boxes.

2. Given `MOV R2, (5000H) // R2 = Mem[Mem[5000]]`, draw the two-hop box diagram showing what happens at memory location `5000H` and beyond, assuming `Mem[5000] = 6100H` and `Mem[6100] = 77`.

3. For the instruction `ADD R1, (20A6H)`, explain in your own words why **two** memory accesses are required, and identify what is retrieved at each step.

4. Write one instruction using **direct** addressing and one using **indirect** addressing that both eventually add a value to R4 from address `1A00H` (direct) and via pointer `1A00H` (indirect). Draw both instruction formats side by side and explain the difference in execution.

5. A system uses direct addressing with only 12 bits for the address field, limiting it to 4096 memory locations. Explain, using the concept of indirect addressing, how this same system could access memory locations beyond that 4096 limit — draw a box diagram to support your answer.

1. Subtract operand pointed to by address `3B20H` from R3

```
SUB R3, (3B20H)   // R3 = R3 - Mem[Mem[3B20]]
```

Instruction format:

```
| opcode: SUB | register: R3 | address: 3B20H |
```

Execution boxes:

```
Instruction: [ SUB | R3 | 3B20H ]
                          |
                          | (1st access)
                          v
                   Mem[3B20H] = 9010H   <-- pointer, not the operand
                          |
                          | (2nd access)
                          v
                   Mem[9010H] = 20      <-- actual operand

Result: R3 = R3 - 20
```

2. `MOV R2, (5000H) // R2 = Mem[Mem[5000]]`, given `Mem[5000] = 6100H`, `Mem[6100] = 77`

```
Instruction: [ MOV | R2 | 5000H ]
                          |
                          | (1st access: read Mem[5000])
                          v
                   Mem[5000H] = 6100H   <-- this is a pointer, not data
                          |
                          | (2nd access: read Mem[6100])
                          v
                   Mem[6100H] = 77      <-- this IS the operand

Result: R2 = 77
```

3. Why does `ADD R1, (20A6H)` need two memory accesses?

Because the address `20A6H` given in the instruction doesn't point directly to the operand — it points to a memory cell that itself contains *another* address.

- **Step 1 (1st access):** CPU reads `Mem[20A6H]`. This retrieves a value, but that value is actually another memory address (a pointer), not usable data yet.
- **Step 2 (2nd access):** CPU uses that retrieved address and reads `Mem[that address]`. *This* is where the real operand value is stored.

Only after this second lookup does the CPU have the actual number to add to R1. This is exactly why indirect addressing is inherently slower than direct addressing — it needs one extra "hop" through memory.

4. Direct vs. Indirect — adding to R4 via address `1A00H`

Direct addressing:

```
ADD R4, 1A00H   // R4 = R4 + Mem[1A00]
| opcode: ADD | register: R4 | address: 1A00H |
```

Execution: 1 memory access — `Mem[1A00H]` directly holds the operand.

Indirect addressing:

```
ADD R4, (1A00H)   // R4 = R4 + Mem[Mem[1A00]]
| opcode: ADD | register: R4 | address: 1A00H |
```

(same instruction format, but parentheses signal indirection)

Execution: 2 memory accesses:

```
Mem[1A00H] = 2C55H   <-- 1st access gives a pointer
Mem[2C55H] = 18       <-- 2nd access gives the operand
```

Difference: In direct addressing, `Mem[1A00H]` *is* the operand (18, for example). In indirect addressing, `Mem[1A00H]` only *points to* where the operand actually lives — an extra step is needed to reach it.

5. 12-bit direct address field limited to 4096 locations — how indirect addressing extends this

With direct addressing:

```
| opcode | address (12 bits) |
```

Maximum reachable locations = $2^{12} = 4096$. The instruction's address field is *physically incapable* of representing any address beyond location 4095.

With indirect addressing, the instruction's 12-bit field still only points to one of those 4096 locations — but what's *stored* at that location isn't restricted to 12 bits. A memory cell can be, say, 32 bits wide, so it can hold an address far larger than 4096 would allow.

```
Instruction: [ opcode | address (12 bits, max reaches location 4095) ]
                              |
                              | (1st access: within the 4096 limit)
                              v
                       Mem[location ≤ 4095] = 00F45210H   <-- a full 32-bit
address!
                              |
                              | (2nd access: NOT limited to 12 bits)
                              v
                       Mem[00F45210H] = operand
```

Conclusion: The instruction field only needs to point to a *pointer's storage location* (which is within the small 4096 range), but the pointer itself — being ordinary data stored in a normal-width memory cell — can encode addresses far beyond that 12-bit limit. This is exactly why the slide says indirect addressing is "not limited by the number of bits in operand address like direct addressing."

Register Addressing

The core idea: **the operand isn't in memory at all — it's sitting directly in one of the CPU's registers, and the instruction just tells the CPU *which* register to use.**

How it differs from the modes you've seen so far

- **Immediate**: operand value is *inside* the instruction.
- **Direct**: instruction holds a memory *address* → 1 memory access.
- **Indirect**: instruction holds an address of an address → 2 memory accesses.
- **Register**: instruction holds a *register number* → **0 memory accesses**. The CPU just reads straight from its own internal register file.

Key characteristics from the slide

- **The instruction specifies a register number**, and the operand is whatever value is currently sitting in that register.
- **Very few bits needed**: since CPUs typically only have a small number of registers (say, 16 or 32), you only need a handful of bits to identify one — e.g., 5 bits can address up to 32 registers. Compare this to direct addressing, where you might need 16+ bits just to cover the address space of memory. This is why register addressing is very compact.
- **Faster execution**: registers are built directly into the CPU and are far faster to access than main memory. Since there's no memory access at all, this is even faster than direct addressing (which needs one memory access) or indirect addressing (which needs two).
- **Modern load-store architectures support a large number of registers**: this connects back to the very first slide you looked at — load-store ISAs (like RISC processors) keep computation confined mostly to registers, and only use dedicated load/store instructions to move data between registers and memory. Having more registers available means more operands can be handled this fast way, reducing how often the CPU needs to touch memory at all.

Walking through the examples

`ADD R1, R2, R3 // R1 = R2 + R3`

- `ADD` = opcode
- All three operands — `R1`, `R2`, `R3` — are registers
- Execution: CPU reads the values currently held in R2 and R3 (both instantly, no memory access), adds them, and writes the result into R1
- This is a classic three-operand register-only instruction, typical of RISC/load-store architectures

`MOV R2, R5 // R2 = R5`

- `MOV` = opcode (move/copy)
- Execution: CPU simply copies whatever value is in R5 into R2
- Again — entirely within the register file, no memory involved

Mental model

Think of registers like items already sitting in your hands or your pockets — right there, instantly accessible. Direct/indirect addressing is like having to walk to a locker (or worse, a locker whose key is in another locker) to get the item. Register addressing skips all of that: the CPU doesn't need to "go" anywhere, because the value is already close by.

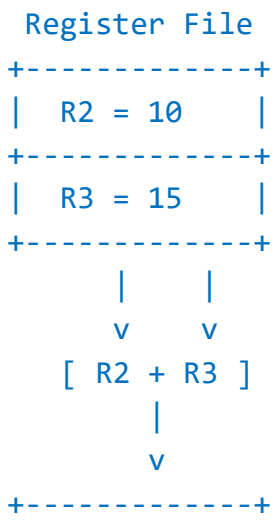
Box Diagrams — Register Addressing

1. `ADD R1, R2, R3 // R1 = R2 + R3`

Instruction format:

```
+---------+------+------+------+
| opcode: | R1   | R2   | R3   |
|  ADD    |(dest)|(src1)|(src2)|
+---------+------+------+------+
```

Execution flow (no memory access — everything stays in the register file):



```
              |  R1 = 25    |  <-- result written back
              +-------------+
```

2. `MOV R2, R5 // R2 = R5`

Instruction format:

```
+---------+------+------+
| opcode: | R2   | R5   |
|  MOV    |(dest)|(src) |
+---------+------+------+
```

Execution flow:

```
               Register File
              +-------------+
              |  R5 = 40    |
              +-------------+
                    |
                    | (copy value)
                    v
              +-------------+
              |  R2 = 40    |  <-- R2 now holds a copy of R5's value
              +-------------+
```

Notice both diagrams have **zero arrows pointing to memory** — everything happens inside the register file, which is exactly why register addressing needs no memory access and is the fastest mode covered so far.

Here's a more complex example — a short sequence of register-addressing instructions working together, like a small computation a compiler might generate.

Complex Example: Computing `R4 = (R1 + R2) - R3`

Since a single instruction typically only has 3 register operands (one destination + two sources), a more complex expression needs to be broken into **multiple register-addressing instructions**, using a temporary register to hold intermediate results.

```
ADD R5, R1, R2   // R5 = R1 + R2   (temporary result)
SUB R4, R5, R3   // R4 = R5 - R3   (final result)
```

Instruction formats

```
Instruction 1:
+---------+------+------+------+
| opcode: | R5   | R1   | R2   |
|  ADD    |(dest)|(src1)|(src2)|
+---------+------+------+------+
```

```
Instruction 2:
+---------+------+------+------+
| opcode: | R4   | R5   | R3   |
|  SUB    |(dest)|(src1)|(src2)|
+---------+------+------+------+
```

Execution flow (box diagram, step by step)

Suppose R1 = 8, R2 = 5, R3 = 4.

```
Step 1: ADD R5, R1, R2

        Register File (before)
        +-------------+
        |  R1 = 8     |
        +-------------+
        |  R2 = 5     |
        +-------------+
              |    |
              v    v
           [ R1 + R2 ]
                |
                v
        +-------------+
        |  R5 = 13    |   <-- new value written
        +-------------+
Step 2: SUB R4, R5, R3

        Register File (after step 1)
        +-------------+
        |  R5 = 13    |
        +-------------+
        |  R3 = 4     |
        +-------------+
              |    |
              v    v
           [ R5 - R3 ]
                |
                v
        +-------------+
        |  R4 = 9     |   <-- final result
        +-------------+
```

Why this matters

- Notice that **no memory is touched at any point** — R5 is used purely as a scratch/temporary register that lives inside the register file, not in main memory.

- This mirrors exactly how real compilers work: a complex expression like `(a + b) - c` gets broken down into a **sequence of simple register-to-register operations**, each using at most 2 source registers and 1 destination register.
- This is the defining style of **load-store / RISC architectures** — complex expressions aren't computed in one giant instruction; they're built up step-by-step, entirely within fast registers, only touching memory when a `LOAD` or `STORE` instruction explicitly asks for it.

Extra complexity: what if the values start in memory?

A fully realistic version would combine register addressing *with* load/store instructions:

```
LOAD  R1, 2000H   // R1 = Mem[2000]   (bring value into register first)
LOAD  R2, 2004H   // R2 = Mem[2004]
ADD   R5, R1, R2  // R5 = R1 + R2     (register addressing)
LOAD  R3, 2008H   // R3 = Mem[2008]
SUB   R4, R5, R3  // R4 = R5 - R3     (register addressing)
STORE R4, 200CH   // Mem[200C] = R4  (store final result back)
```

This shows the typical pattern: **load data into registers → do all the computation using register addressing (fast, no memory) → store the final result back to memory.** Only the `LOAD`/`STORE` instructions touch memory; every arithmetic step in between is pure register addressing.

Questions

1. Write a sequence of register addressing instructions to compute `R6 = (R1 × R2) + R3`, using a temporary register for the intermediate result. Draw the instruction formats as boxes.

2. Given `R1 = 12`, `R2 = 3`, `R3 = 7`, trace through the instructions `MUL R5, R1, R2` followed by `ADD R6, R5, R3` using box diagrams showing the register file at each step.

3. Write a sequence of instructions to compute `R7 = (R1 - R2) × (R3 + R4)`. How many temporary registers are needed, and why?

4. Draw the instruction format (boxes) for `MOV R3, R7`, and explain what happens to the register file during execution.

5. Write instructions to swap the values of R1 and R2 using only register addressing (no memory access), using one temporary register. Draw each step's effect on the register file.

6. Given the sequence:

```
ADD R5, R1, R2
SUB R6, R5, R3
MOV R7, R6
```

Draw a box diagram tracing the values through all three instructions, assuming R1 = 20, R2 = 10, R3 = 5.

7. Write a full instruction sequence (including `LOAD`/`STORE`) that computes `Mem[3000] = R1 + R2 - R3`, where R1, R2, R3 already hold values, and explain which instructions use register addressing and which use direct addressing.

8. Why can't a single register addressing instruction typically compute an expression like `(R1 + R2) - R3` directly? Explain in terms of instruction format limitations.

9. Write instructions to compute `R4 = R1 + R2 + R3` (three-way sum) using only two-source-register instructions. Draw the box diagram showing each addition step.

10. Given a load-store architecture, explain (with a short instruction sequence and box diagram) why increasing the number of available registers reduces the need for `LOAD`/`STORE` instructions in a computation.

Model Answers

1. `R6 = (R1 × R2) + R3`

```
MUL R5, R1, R2   // R5 = R1 × R2
ADD R6, R5, R3   // R6 = R5 + R3
```

2. Trace with R1=12, R2=3, R3=7

```
Step 1: MUL R5, R1, R2
  R1=12 --\
           [ R1 × R2 ] --> R5 = 36
  R2=3  --/

Step 2: ADD R6, R5, R3
  R5=36 --\
           [ R5 + R3 ] --> R6 = 43
  R3=7  --/
```

3. `R7 = (R1 - R2) × (R3 + R4)`

```
SUB R5, R1, R2   // R5 = R1 - R2
ADD R6, R3, R4   // R6 = R3 + R4
MUL R7, R5, R6   // R7 = R5 × R6
```

Two temporary registers needed (R5 and R6) — because each register instruction only has 2 source operands, and this expression has two separate sub-results (the subtraction and the addition) that must both be computed *before* they can be combined.

4. `MOV R3, R7`

```
+------+------+------+
| MOV  |  R3  |  R7  |
+------+------+------+
Register File (before)          Register File (after)
+-----------+                   +-----------+
| R7 = 50   |                   | R7 = 50   |
+-----------+        -->        +-----------+
| R3 = ??   |                   | R3 = 50   |  <-- copied from R7
+-----------+                   +-----------+
```

R3's old value is overwritten with a copy of R7's value; R7 itself is unchanged.

5. Swap R1 and R2 using a temp register

```
MOV R3, R1   // R3 = R1  (save R1)
MOV R1, R2   // R1 = R2
MOV R2, R3   // R2 = R3  (restore original R1 into R2)
Step 0:  R1=5, R2=9, R3=?
Step 1 (MOV R3,R1):  R3=5   -> R1=5, R2=9, R3=5
Step 2 (MOV R1,R2):  R1=9   -> R1=9, R2=9, R3=5
Step 3 (MOV R2,R3):  R2=5   -> R1=9, R2=5, R3=5
```

Final: R1=9, R2=5 — successfully swapped.

6. Trace with R1=20, R2=10, R3=5

```
ADD R5, R1, R2   // R5 = 20+10 = 30
+------+------+
| R1=20| R2=10|
+------+------+
     [ + ]
      R5 = 30

SUB R6, R5, R3   // R6 = 30-5 = 25
+------+------+
| R5=30| R3=5 |
+------+------+
     [ - ]
      R6 = 25

MOV R7, R6       // R7 = 25
+------+
| R6=25|
+------+
```

```
[copy]
R7 = 25
```

7. `Mem[3000] = R1 + R2 - R3`

```
ADD R5, R1, R2     // register addressing: R5 = R1 + R2
SUB R6, R5, R3     // register addressing: R6 = R5 - R3
STORE R6, 3000H    // direct addressing: Mem[3000] = R6
```

- `ADD` and `SUB` use **register addressing** (all operands are registers, no memory access).
- `STORE` uses **direct addressing** for the destination (`3000H` is a memory address) combined with register addressing for the source (R6).

8. Why can't one instruction compute `(R1 + R2) - R3` directly?

Because a standard register addressing instruction format only has room for a fixed, small number of operand fields — typically one destination and two source registers:

```
| opcode | dest | src1 | src2 |
```

There's no field left to hold a *third* source register. So an expression needing three input values must be broken into two instructions, using an intermediate register to carry the partial result from the first instruction into the second.

9. `R4 = R1 + R2 + R3`

```
ADD R5, R1, R2   // R5 = R1 + R2
ADD R4, R5, R3   // R4 = R5 + R3
Step 1: [R1] + [R2] --> R5
Step 2: [R5] + [R3] --> R4
```

10. More registers → fewer LOAD/STORE instructions

With few registers, values must constantly be spilled to memory and reloaded:

```
LOAD  R1, 2000H
LOAD  R2, 2004H
ADD   R3, R1, R2
STORE R3, 2008H   // must store because registers are needed for next values
LOAD  R1, 200CH
...
```

With more registers available, all intermediate values can stay in registers simultaneously:

```
LOAD R1, 2000H
LOAD R2, 2004H
LOAD R3, 2008H
```

```
LOAD R4, 200CH
ADD  R5, R1, R2
ADD  R6, R3, R4
ADD  R7, R5, R6
STORE R7, 3000H   // only one store needed at the very end
Register File (many registers available)
+----+----+----+----+----+----+----+
| R1 | R2 | R3 | R4 | R5 | R6 | R7 |
+----+----+----+----+----+----+----+
   all intermediate values held simultaneously — no need to spill to memory
```

Fewer LOAD/STORE instructions means fewer (slow) memory accesses overall, which is exactly why modern load-store architectures are designed with a large register file — it lets more computation happen using fast register addressing instead of repeatedly touching memory.

Register Indirect Addressing

The core idea: **the instruction specifies a register, but that register doesn't hold the operand itself — it holds a memory address, and the operand is stored at that address.**

How it fits with what you've learned

- **Register addressing**: register holds the operand *directly*.
- **Direct addressing**: instruction field holds a memory *address* directly.
- **Indirect addressing**: instruction field holds an address, which points to *another* address, which finally points to the operand (2 memory accesses).
- **Register indirect addressing**: instruction field holds a *register number*, and that register holds the memory address of the operand (**1 memory access**).

So it's essentially a hybrid: it uses a register the way register addressing does (to name where to look), but what's in that register is a *pointer to memory*, not the value itself.

Key characteristics from the slide

- **The register holds the memory address**, not the operand — so the CPU first reads the register (fast, no memory access) to get an address, then makes **one memory access** to fetch the actual operand from that address.
- **Can access a large address space**: since the address is stored in a full-width register (e.g., 32 or 64 bits), it isn't limited by the small number of bits available in the instruction's address field — unlike direct addressing, which is capped by however many bits the instruction format sets aside.
- **One fewer memory access than indirect addressing**: indirect addressing needs 2 memory accesses (instruction → address in memory → another address → operand). Register indirect skips the first memory access entirely because the "first hop" is a register (already in the CPU), not memory. So it only needs **1** memory access to reach the operand.

Walking through the example

```
ADD R1, (R5) // R1 = R1 + Mem[R5]
```

(Note: the slide shows "PC = R1 + Mem[R5]" — this looks like a typo; based on the pattern from earlier addressing modes, it should be `R1 = R1 + Mem[R5]`.)

- `ADD` = opcode
- `R1` = destination register (register addressing — receives the result, and is also a source operand)
- `(R5)` = register indirect — the parentheses indicate "the value in R5 is not the operand; it's the address of the operand"

Execution:

1. CPU reads the value currently in R5 — say R5 contains `3000H`.
2. CPU uses `3000H` as a memory address and accesses `Mem[3000H]` — this is the actual operand, say `18`.
3. CPU adds 18 to R1 and stores the result back in R1.

More examples

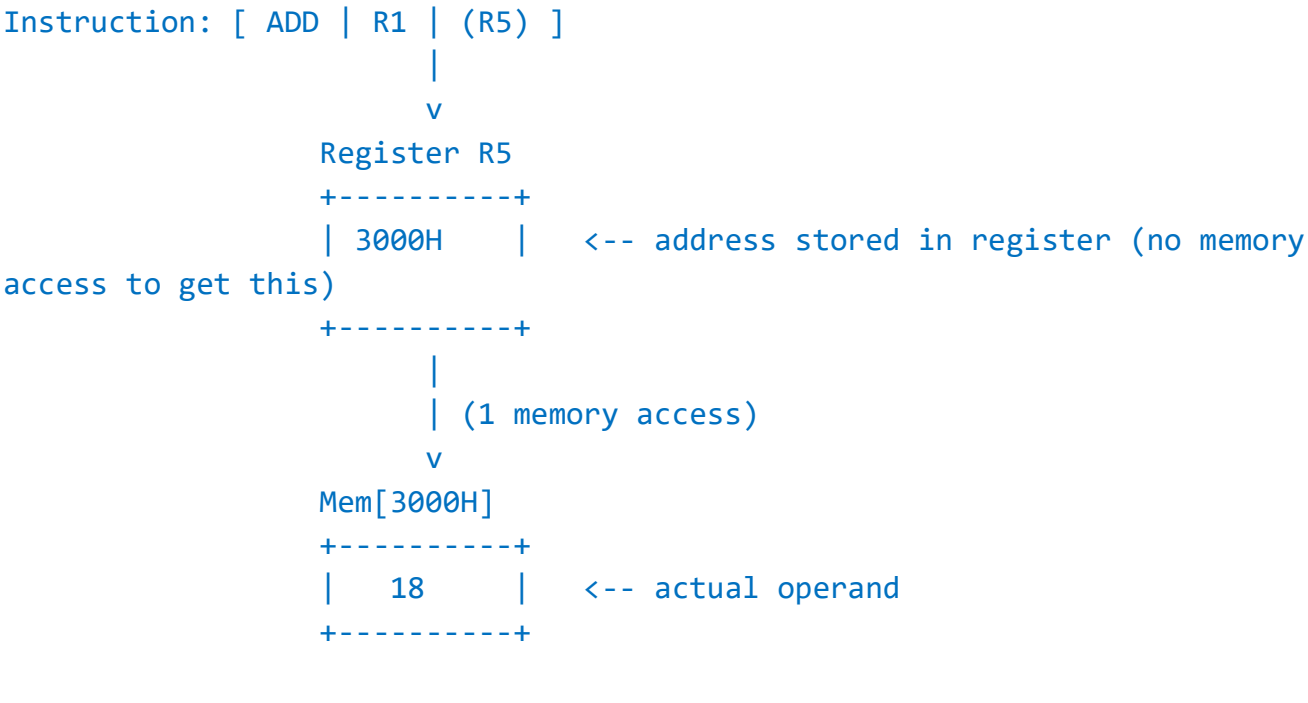
`MOV R2, (R4) // R2 = Mem[R4]`

- R4 holds a memory address (say `5A00H`)
- CPU fetches `Mem[5A00H]` and loads that value into R2

`SUB R3, (R6) // R3 = R3 - Mem[R6]`

- R6 holds an address (say `7000H`)
- CPU fetches `Mem[7000H]` and subtracts it from R3

Box diagram



Result: R1 = R1 + 18

Mental model

Think of it like a friend handing you a **key** (the register) that opens a specific **locker** (the memory address stored in that register) — you don't need directions to find the key, since it's already in your hand. You just use the key once to open the locker and grab what's inside. Compare this to indirect addressing, where you'd first need to go find the key itself in some other locker before you could even use it.

need questions and answers for this too need more complex questions

Architected multi-layered questions with visual aids and practical applications
Architected multi-layered questions with visual aids and practical applications

Questions — Register Indirect Addressing (Complex)

1. Given `ADD R1, (R5)` where R5 = `3000H` and `Mem[3000H] = 25`, and initially R1 = 10, trace the execution step-by-step and draw a box diagram. What is the final value of R1?

2. Write a register indirect instruction that stores the value in R2 into the memory location pointed to by R6. Draw its instruction format and execution boxes.

3. Compare direct, indirect, and register indirect addressing for the instruction "add the value at memory location X to R1." Write one instruction for each mode and state how many memory accesses each requires.

4. A pointer to an array's first element is stored in R7. Write a register indirect instruction to load the first array element into R1. Then explain why this pattern is useful for traversing arrays, compared to direct addressing.

5. Given the sequence:

```
MOV R5, 4000H      // R5 = 4000H (load address into register)
ADD R1, (R5)       // R1 = R1 + Mem[R5]
```

Explain what each instruction does, which addressing modes are used in each, and draw a combined box diagram tracing both steps. Assume R1 = 5 initially and `Mem[4000H] = 30`.

6. Why does register indirect addressing require **one fewer** memory access than indirect addressing, even though both ultimately rely on a "pointer to find the operand"? Explain the difference using box diagrams for both.

7. Write two instructions that swap the values pointed to by R1 and R2 (i.e., swap `Mem[R1]` and `Mem[R2]`) using register indirect addressing and one temporary register R3. Draw the box diagram for each step.

8. A linked list node stores its "next" pointer in R4. Write a register indirect instruction to load the next node's address into R5 (i.e., `R5 = Mem[R4]`). Explain, in the context of large address spaces, why register indirect addressing is well-suited for traversing data structures like linked lists.

9. Given `SUB R2, (R3) // R2 = R2 - Mem[R3]`, suppose R3 = `8100H` and `Mem[8100H] = 12`, and R2 = 50 initially. Trace the full execution with a box diagram and state the final value of R2.

10. Explain, using a box diagram, why register indirect addressing "can access a large address space" even though the *instruction* only reserves a few bits for specifying the register (e.g., 4 bits for 16 registers).

Model Answers

1. `ADD R1, (R5)`, R5=3000H, Mem[3000H]=25, R1=10

```
Register R5 = 3000H  (read directly, no memory access)
        |
        | (1 memory access)
        v
Mem[3000H] = 25    <-- operand
        |
        v
R1 = R1 + 25 = 10 + 25 = 35
```

Final R1 = 35

2. Store R2 into location pointed to by R6

```
STORE R2, (R6)   // Mem[R6] = R2
+---------+------+--------+
| opcode: |  R2  | (R6)   |
| STORE   |(src) |(pointer)|
+---------+------+--------+
Register R6 = address A (read from register, no memory access)
        |
        v
Mem[A] = value in R2   (1 memory access – write, not read)
```

3. Direct vs. Indirect vs. Register Indirect — "add Mem[X] to R1"

Direct:

```
ADD R1, X   // R1 = R1 + Mem[X]
```

1 memory access (X is right in the instruction).

Indirect:

```
ADD R1, (X)   // R1 = R1 + Mem[Mem[X]]
```

2 memory accesses (X → pointer → operand).

Register Indirect (assume R5 already holds X):

```
ADD R1, (R5)   // R1 = R1 + Mem[R5], where R5 = X
```

1 memory access (reading R5 is free; only Mem[X] costs an access).

Key insight: register indirect achieves the same single-access cost as direct addressing, but without being limited by the instruction's address field width, since X lives in a full-width register instead of the instruction itself.

4. Array traversal using register indirect

```
MOV R1, (R7)   // R1 = Mem[R7]   (R7 holds address of array[0])
```

Execution: R7 already contains the array's base address; the CPU reads Mem[R7] to get the first element directly into R1.

Why useful for traversal: After reading the element, you can simply do `ADD R7, R7, #1` (or `#4` for word-sized elements) to advance the pointer, then repeat `MOV R1, (R7)` to get the next element. Direct addressing can't do this efficiently because the address is baked into the instruction and can't be changed at runtime — you'd need a different instruction for every array index. With register indirect, the *same* instruction works for every element; only the register's contents change.

5. `MOV R5, 4000H` then `ADD R1, (R5)`

- Instruction 1: `MOV R5, 4000H` — uses **immediate addressing** (the literal value `4000H` is loaded straight into R5; no memory access).
- Instruction 2: `ADD R1, (R5)` — uses **register indirect addressing** (R5 holds an address; CPU accesses memory once to get the operand).

```
Step 1: MOV R5, 4000H
   Instruction: [ MOV | R5 | #4000H ]
                              |
                              v
                     R5 = 4000H   (no memory access)

Step 2: ADD R1, (R5)
   Register R5 = 4000H
        |
        | (1 memory access)
        v
   Mem[4000H] = 30
        |
        v
   R1 = R1 + 30 = 5 + 30 = 35
```

Final R1 = 35

6. Why register indirect needs one fewer access than indirect

Indirect addressing:

```
Instruction --> [address A in instruction]
                          |
                          | (1st memory access: get Mem[A] = address B)
                          v
                    Mem[A] = B
                          |
                          | (2nd memory access: get Mem[B] = operand)
                          v
                    Mem[B] = operand
```

Both "hops" are memory accesses because the *first* address (A) must itself be fetched from memory.

Register indirect addressing:

```
Instruction --> [register number, e.g. R5]
                          |
                          | (read register — NOT a memory access, it's internal
to CPU)
                          v
                    R5 = address A
                          |
                          | (1 memory access: get Mem[A] = operand)
                          v
                    Mem[A] = operand
```

The first "hop" (getting address A) comes from a **register**, which the CPU can read instantly without touching memory. Only the second hop touches memory. That's the one access saved.

7. Swap `Mem[R1]` and `Mem[R2]` using register indirect + temp register R3

```
MOV R3, (R1)   // R3 = Mem[R1]        (save value at R1's address)
MOV R4, (R2)   // R4 = Mem[R2]        (read value at R2's address)
STORE R4, (R1) // Mem[R1] = R4        (write R2's old value into R1's address)
STORE R3, (R2) // Mem[R2] = R3        (write R1's old value into R2's address)
Before: Mem[R1]=100, Mem[R2]=200

Step 1: R3 = Mem[R1] = 100
Step 2: R4 = Mem[R2] = 200
Step 3: Mem[R1] = R4 = 200   --> Mem[R1] now 200
Step 4: Mem[R2] = R3 = 100   --> Mem[R2] now 100

After: Mem[R1]=200, Mem[R2]=100  -- swapped
```

8. Linked list traversal — `R5 = Mem[R4]` (next pointer)

```
MOV R5, (R4)   // R5 = Mem[R4]
```

```
Register R4 = address of current node   (read instantly, no memory access)
        |
        | (1 memory access — the "next" field of the node)
        v
Mem[R4] = address of next node
        |
        v
R5 = address of next node
```

Why well-suited for large address spaces: Linked list nodes can be scattered anywhere across a large memory space — their addresses aren't predictable or fixed at compile time. Since R4 (a full-width register) can hold *any* valid memory address, register indirect addressing lets the program follow pointers wherever they lead, without being restricted by a small address field in the instruction itself (as direct addressing would be). This is exactly why pointer-heavy structures like linked lists, trees, and dynamically allocated data rely on register indirect addressing.

9. `SUB R2, (R3)`, R3=8100H, Mem[8100H]=12, R2=50

```
Register R3 = 8100H   (no memory access)
        |
        | (1 memory access)
        v
Mem[8100H] = 12
        |
        v
R2 = R2 - 12 = 50 - 12 = 38
```

Final R2 = 38

10. Why register indirect reaches a large address space despite few bits for the register field

```
Instruction: [ opcode | register: 4 bits (selects 1 of 16 registers) ]
                                |
                                v
                    Register file (16 registers)
                    +----------------------+
                    | R5 = 00F45210H       |  <-- full-width value, e.g. 32
bits
                    +----------------------+
                                |
                                v
                    Mem[00F45210H] = operand
```

The instruction's 4 bits only need to *select which register* to look at (1 of 16) — they don't need to encode the address itself. The actual address is stored **inside** that register, which is typically as wide as the

machine's word size (32 or 64 bits). So the reachable address space is limited by the register's width, not by the tiny number of bits spent identifying which register to use. This is the same underlying trick as indirect addressing (storing the "real" address in memory instead of the instruction) — register indirect just stores that pointer in a register instead, saving a memory access in the process.

Relative Addressing (PC-Relative)

The core idea: **instead of specifying an absolute memory address, the instruction specifies an *offset* (displacement) that gets added to the Program Counter (PC) to calculate the effective address of the operand.**

How it differs from what you've seen so far

- **Direct addressing**: instruction holds the *actual* address of the operand.
- **Relative addressing**: instruction holds a small *offset* — a signed number of locations *away from* the current instruction — and the CPU calculates the real address by adding this offset to the PC.

So the "address" isn't fixed in the instruction; it's computed dynamically relative to wherever execution currently is.

Key characteristics from the slide

- **Effective address = PC + offset**: the PC holds the address of the current (or next) instruction, and the offset tells the CPU how far away the operand is *relative to that point*.
- **Limited range**: since the offset field has a fixed number of bits, it can only represent a limited span of displacement — you can't jump arbitrarily far away.
- **12-bit offset example**: with 12 bits used as a **signed** value (two's complement), the range is from **-2048 to +2047**. That's $2^{12} = 4096$ total representable values, split between negative (backward) and positive (forward) displacements.

Reading the diagram

```
| opcode | offset |  --\
                        [ ADDER ] ---> effective address ---> Memory[effective
address] = operand
| PC               |  --/
```

- The **opcode + offset** field comes from the instruction itself.
- The **PC** (current instruction address) comes from the CPU.
- Both feed into an **ADDER**, which computes: `effective address = PC + offset`.
- That computed address is then used to access **memory**, retrieving the operand.

Why this mode is useful

- **Compact instructions**: since the offset only needs to cover a small nearby range, it fits in far fewer bits than a full absolute address would.

- **Position-independent code**: because the address is always calculated *relative to* wherever the program currently is, the same code can be loaded at different memory locations and still work correctly — this is a huge advantage for relocatable programs and shared libraries.
- **Very common for branch/jump instructions**: relative addressing is the standard way CPUs implement conditional branches and loops (e.g., "jump 10 instructions back to repeat this loop") — the target is naturally expressed relative to the current position.

Harder Questions

1. Given `PC = 5000H` and offset `= -16` (decimal), calculate the effective address in hex. Show your work.

2. A CPU uses a 10-bit signed offset field for relative addressing. Calculate the minimum and maximum offset values it can represent, and explain how you derived them.

3. Explain, using the box/adder diagram, why relative addressing is well-suited for **position-independent code** but direct addressing is not. Use a concrete example where a program is loaded at two different starting addresses.

4. A branch instruction `BEQ offset` is at memory address `2000H`. If the target instruction it needs to jump to is at `1FF0H`, calculate the offset value that must be encoded in the instruction (assume PC = address of the *next* instruction, i.e., `2004H` for a 4-byte instruction).

5. Explain why relative addressing's range is described as **limited**, even though the PC itself can point anywhere in a large address space. What exactly is limited, and what isn't?

6. A 12-bit offset field ranges from -2048 to +2047. Suppose a program needs to branch to a target that is 3000 instructions away. Explain why this is a problem for relative addressing, and describe one general strategy compilers/assemblers use to work around this limitation.

7. Compare relative addressing and register indirect addressing in terms of: (a) what determines the effective address, (b) whether the "base" value changes between different runs/loads of the program, and (c) typical use case for each.

8. Draw a box diagram (opcode/offset/PC/adder/memory) for the instruction `LOAD R1, [PC+24]`, assuming `PC = 6000H`. Calculate the effective address and show what value would be loaded if `Mem[6018H] = 99`.

9. Explain why relative addressing instructions are typically **shorter** (fewer bits) than direct addressing instructions, even though both ultimately access memory. Use the 12-bit offset example to support your answer numerically.

10. A loop body is 40 instructions long, and each instruction is 4 bytes. Write the offset value (in decimal) needed for a branch instruction at the *end* of the loop to jump back to the *start* of the loop, assuming PC-relative addressing where PC = address of the branch instruction + 4 (i.e., the next instruction).

Model Answers

1. PC = 5000H, offset = -16

```
5000H = 20480 decimal
Effective address = 20480 + (-16) = 20464 decimal = 4FF0H
```

Effective address = 4FF0H

2. 10-bit signed offset — range

With 10 bits in two's complement:

- Total representable values = $2^{10} = 1024$
- Range splits as: **-512 to +511**
- Derivation: half the values (512) represent negative numbers (-1 to -512), and the other half (512) represent zero and positive numbers (0 to 511).

3. Relative addressing and position-independent code

```
Program loaded at address 1000H:
Instruction at 1000H: BRANCH offset=+20H  --> effective address = PC(1004H) +
20H = 1024H

Same program loaded at address 5000H:
Instruction at 5000H: BRANCH offset=+20H  --> effective address = PC(5004H) +
20H = 5024H
```

Because the effective address is always computed as `PC + offset`, and the offset value inside the instruction never changes, the *same* instruction correctly reaches "20H bytes ahead of wherever I currently am" — regardless of where in memory the program was loaded.

With **direct addressing**, the instruction would hardcode an absolute address like `1024H`. If the program is then loaded at `5000H` instead of `1000H`, that hardcoded address `1024H` would be wrong — it would need to be `5024H` instead. This means direct addressing requires the program to be modified (relocated) for each different loading address, while relative addressing does not.

4. Branch at 2000H, target at 1FF0H, PC = 2004H (next instruction)

```
Effective address = PC + offset
1FF0H = 2004H + offset
offset = 1FF0H - 2004H
```

Convert to decimal: `1FF0H = 8176`, `2004H = 8196`

```
offset = 8176 - 8196 = -20 (decimal)
```

Offset = -20 (a backward branch, which makes sense since the target 1FF0H is before 2004H).

5. What exactly is limited?

What's **limited**: the *distance* (displacement) between the current instruction and the target — because the offset field has a fixed, small number of bits (e.g., 12 bits → max ±2047/±2048 locations away).

What's **not limited**: the *absolute* address the PC itself can point to — the PC is typically a full-width register (32/64 bits), so it can represent addresses across the entire memory space. Relative addressing doesn't restrict *where in memory* the code can run; it only restricts *how far away* (in either direction) a single instruction's target can be from where execution currently is.

6. Branching 3000 instructions away with a 12-bit offset (max ±2047)

Problem: 3000 exceeds the maximum representable offset of 2047, so a single relative-addressing instruction physically cannot encode a jump that far — the offset field would overflow.

Workaround strategy: Assemblers/compilers use a technique often called **branch expansion** (or "long branch" / "trampoline"): instead of one direct relative branch, they generate a short relative branch to a nearby intermediate instruction, which then performs an **absolute** (or indirect) jump to the actual far-away target. For example:

```
BEQ near_target        // short relative branch (within range)
...
near_target:
JMP far_target_absolute // unconditional absolute jump covers the full distance
```

This effectively "hops" through an intermediate point to cover distances beyond what the offset field alone can reach.

7. Relative vs. Register Indirect

Aspect	Relative Addressing	Register Indirect Addressing
(a) Determines effective address	`PC + offset` (offset comes from the instruction)	Value stored in a specified register
(b) Does the "base" change between runs?	PC naturally changes as the program is loaded/executed, but the offset stays fixed — works correctly regardless of load address	Register content is set explicitly by prior instructions (e.g., `MOV`, `LOAD`) — must be initialized correctly each time before use
(c) Typical use case	Branches, loops, position-independent code	Pointer dereferencing, array traversal, linked list traversal, dynamically allocated data

8. `LOAD R1, [PC+24]`, PC = 6000H

```
Instruction: [ opcode: LOAD | R1 | offset: 24 ]
                                        |
                                        v
                                  [ ADDER ]  <-- also receives PC = 6000H
                                        |
                                        v
                          effective address = 6000H + 24 (decimal, 0x18)
                                        = 6018H
                                        |
                                        v
                                  Mem[6018H] = 99
                                        |
                                        v
                                  R1 = 99
```

Effective address = 6018H; R1 is loaded with 99.

9. Why relative addressing instructions are shorter than direct addressing instructions

Direct addressing needs an address field wide enough to represent *every possible location in the entire memory space* — e.g., for a 1MB memory, that's 20 bits.

Relative addressing only needs a field wide enough to represent the *maximum expected distance* between an instruction and its typical target (e.g., a nearby branch target) — which is usually much smaller. With a 12-bit offset, only 12 bits are needed to cover a range of -2048 to +2047 locations, compared to 20 bits needed to address the same 1MB space directly.

Numerically: 12 bits (relative) vs. 20 bits (direct, for 1MB) — relative addressing saves 8 bits per instruction in this example, which adds up significantly across a whole program, especially for very common instructions like branches.

10. Loop of 40 instructions, 4 bytes each, branch at end jumps to start

```
Loop start address = X
Loop body = 40 instructions × 4 bytes = 160 bytes
Branch instruction is the 41st instruction (right after the loop body), so:
Branch instruction address = X + 160
PC (next instruction after branch) = X + 160 + 4 = X + 164

Effective address needed = X (jump back to loop start)
offset = effective address - PC = X - (X + 164) = -164
```

Offset = -164 (decimal, a backward branch of 164 bytes to return to the top of the loop).

Converting `5000H` to Decimal — Step by Step

Hexadecimal is a **base-16** number system. Each digit's position represents a power of 16, just like decimal (base-10) uses powers of 10.

`5000H` has 4 hex digits: **5, 0, 0, 0**

Breaking it down by position

Digit	Position (power of 16)	Value
5	16^3	5 × 4096
0	16^2	0 × 256
0	16^1	0 × 16
0	16^0	0 × 1

Calculating each term

```
5 × 16³ = 5 × 4096 = 20480
0 × 16² = 0 × 256   = 0
0 × 16¹ = 0 × 16    = 0
0 × 16⁰ = 0 × 1     = 0
                      -----
Sum:                  20480
```

So `5000H = 20480` in decimal.

(Quick sanity check: in a 32-bit system, an address like this would normally be shown as `00005000H`, padded to 8 hex digits since 32 bits = 8 hex digits. The leading zeros don't change the value — they're just padding to show the full register width.)

The Arithmetic Part

```
Effective address = PC + offset
                  = 20480 + (-16)
                  = 20480 - 16
                  = 20464
```

This is straightforward decimal subtraction — adding a negative offset is the same as subtracting its magnitude.

Converting `20464` Back to Hex

To convert decimal → hex, repeatedly divide by 16 and track the remainders:

```
20464 ÷ 16 = 1279  remainder 0   →  digit: 0
 1279 ÷ 16 =   79  remainder 15  →  digit: F  (15 in hex = F)
   79 ÷ 16 =    4  remainder 15  →  digit: F  (15 in hex = F)
    4 ÷ 16 =    0  remainder 4   →  digit: 4
```

Reading the remainders from **bottom to top** gives the hex digits:

```
4  F  F  0
```

So `20464 = 4FF0H`

Quick verification (reverse check)

```
4 × 16³ = 4 × 4096 = 16384
F × 16² = 15 × 256 = 3840
F × 16¹ = 15 × 16  = 240
0 × 16⁰ = 0 × 1    = 0
                     -----
Sum:                 20464   ✓ matches
```

Note on the 32-bit Standard

In a 32-bit system, addresses and offsets are stored in **32-bit registers**, so this whole calculation would really happen as:

```
PC (32-bit)     = 00005000H
offset (32-bit, sign-extended from a smaller field like 12 bits) = FFFFFFF0H
(this is -16 in 32-bit two's complement)

Effective address = 00005000H + FFFFFFF0H
```

The offset here, `-16` in a 12-bit signed field, gets **sign-extended** to fill the full 32 bits with 1s (since it's negative) before being added — this is standard practice so the addition works correctly using ordinary 32-bit binary addition (where addition automatically handles the "subtraction" via two's complement representation). The result of this 32-bit addition still comes out to `00004FF0H`, matching what we calculated above — the extra leading zeros/Fs just reflect the full register width rather than changing the actual numeric result.

Question 4 — Full Detailed Explanation

Given:

- Branch instruction `BEQ offset` is located at memory address `2000H`
- The target instruction (where we want to jump to) is at `1FF0H`
- PC = address of the *next* instruction after the branch = `2004H` (since instructions are 4 bytes here)
- We need to find: what offset value must be encoded in the instruction?

Step 1: Understand why PC = 2004H, not 2000H

This is a subtle but important point. In most PC-relative architectures, by the time the CPU is *executing* an instruction, the PC has often already been advanced to point to the **next** instruction (this happens during the fetch stage — the CPU increments PC right after fetching, before the instruction even finishes executing).

Since the branch instruction is 4 bytes wide and sits at `2000H`, the next instruction in sequence would naturally start at:

```
2000H + 4 bytes = 2004H
```

So when the CPU computes `effective address = PC + offset`, it uses `PC = 2004H`, not the branch instruction's own address (`2000H`). This is exactly what the question tells us to assume.

Step 2: Convert both addresses to decimal

We need decimal values to do the subtraction cleanly.

Converting `1FF0H` to decimal:

Digit	Position	Value
1	16^3	$1 \times 4096 = 4096$
F	16^2	$15 \times 256 = 3840$
F	16^1	$15 \times 16 = 240$
0	16^0	$0 \times 1 = 0$

```
4096 + 3840 + 240 + 0 = 8176
```

So `1FF0H = 8176` decimal.

Converting `2004H` to decimal:

Digit	Position	Value
2	16^3	$2 \times 4096 = 8192$
0	16^2	$0 \times 256 = 0$
0	16^1	$0 \times 16 = 0$
4	16^0	$4 \times 1 = 4$

```
8192 + 0 + 0 + 4 = 8196
```

So `2004H = 8196` decimal.

Step 3: Set up the relative addressing formula and solve for offset

We know the fundamental relative addressing formula:

```
effective address = PC + offset
```

We want the effective address to equal the target (`1FF0H` = 8176), and we know PC = 8196. So we rearrange to solve for the unknown offset:

```
offset = effective address - PC
offset = 8176 - 8196
offset = -20
```

Why is it negative? Because the target address (`1FF0H` = 8176) comes *before* the PC (`2004H` = 8196) in memory — the branch is jumping *backward*. Any time the target is at a lower address than the PC, the offset must be negative to "walk backward" from PC to reach it. If the target were *ahead* of the PC, the offset would come out positive instead.

Step 4: Sanity-check the answer by working forward

We can verify by plugging the offset back into the formula:

```
effective address = PC + offset
                  = 8196 + (-20)
                  = 8176
```

Converting 8176 back to hex confirms it equals `1FF0H` ✓ — matching our original target. The math checks out.

Step 5: How this offset would actually be encoded in the instruction (bonus insight)

The value **−20** isn't stored as a plain decimal number in hardware — it needs to be represented in **two's complement binary**, since that's how negative numbers are stored in a fixed-width binary field.

If we assume the offset field is, say, 12 bits wide (as in the earlier slide example):

```
+20 in 12-bit binary =  0000 0001 0100
```

To get −20 in two's complement:

1. Invert all the bits: `1111 1110 1011`
2. Add 1: `1111 1110 1100`

So `-20` would be stored in the instruction's offset field as the 12-bit pattern `1111 1110 1100`, which the hardware interprets as −20 when it performs the addition with the PC.

Summary

```
Target address        = 1FF0H = 8176 (decimal)
PC (next instruction)  = 2004H = 8196 (decimal)
```

```
offset = target − PC = 8176 − 8196 = −20
```

```
Verification: PC + offset = 8196 + (−20) = 8176 = 1FF0H ✓
```

Final answer: offset = −20 (a backward branch, since the target address is lower than the PC).

3 More Questions (Same Style as Question 4)

1. A branch instruction `BNE offset` is located at memory address `3100H`. The target instruction it needs to jump to is at `3150H`. Assuming instructions are 4 bytes wide and PC = address of the next instruction, calculate the offset value that must be encoded in the instruction.

2. A `CALL offset` instruction sits at memory address `A200H`. The subroutine it needs to call starts at address `A180H`. Instructions are 4 bytes wide, and PC = address of the next instruction after the call. Calculate the offset, and state whether it represents a forward or backward jump.

3. A conditional branch `BLT offset` is located at address `1000H`. The target label is `100 bytes` after the branch instruction itself (not after the next instruction). Instructions are 4 bytes wide, and PC = address of the next instruction. Calculate the offset value that must be encoded, being careful about how "100 bytes after the branch instruction" relates to PC.

Model Answers

1. BNE at 3100H, target 3150H, PC = next instruction

Step 1 — Find PC:

```
PC = branch address + instruction size = 3100H + 4 = 3104H
```

Step 2 — Convert to decimal:

```
3100H = 3×4096 + 1×256 + 0×16 + 0×1 = 12288 + 256 + 0 + 0 = 12544
3150H = 3×4096 + 1×256 + 5×16 + 0×1 = 12288 + 256 + 80 + 0 = 12624
3104H = 3×4096 + 1×256 + 0×16 + 4×1 = 12288 + 256 + 0 + 4 = 12548
```

Step 3 — Apply the formula:

```
offset = target − PC = 12624 − 12548 = 76
```

Step 4 — Verify:

```
PC + offset = 12548 + 76 = 12624 = 3150H ✓
```

Answer: offset = +76 (positive → forward branch, since the target is ahead of the PC).

2. `CALL` at `A200H`, subroutine at `A180H`, PC = next instruction

Step 1 — Find PC:

```
PC = A200H + 4 = A204H
```

Step 2 — Convert to decimal:

```
A200H = 10×4096 + 2×256 + 0×16 + 0×1 = 40960 + 512 + 0 + 0 = 41472
A180H = 10×4096 + 1×256 + 8×16 + 0×1 = 40960 + 256 + 128 + 0 = 41344
A204H = 10×4096 + 2×256 + 0×16 + 4×1 = 40960 + 512 + 0 + 4 = 41476
```

Step 3 — Apply the formula:

```
offset = target − PC = 41344 − 41476 = −132
```

Step 4 — Verify:

```
PC + offset = 41476 + (−132) = 41344 = A180H ✓
```

Answer: offset = −132 — this is a **backward** jump, since the subroutine address (`A180H`) is lower than the PC (`A204H`).

3. `BLT` at `1000H`, target is 100 bytes after the branch instruction itself, PC = next instruction

This question is trickier because the target is described relative to the **branch instruction's own address**, not relative to PC — so we must first find the actual target address before we can compute the offset relative to PC.

Step 1 — Find the actual target address:

```
Target address = branch instruction address + 100 bytes
               = 1000H + 100 (decimal)

Convert 1000H to decimal: 1×4096 + 0 + 0 + 0 = 4096
Target address (decimal) = 4096 + 100 = 4196

Convert 4196 back to hex:
4196 ÷ 16 = 262 remainder 4   → digit 4
262 ÷ 16  = 16  remainder 6   → digit 6
16 ÷ 16   = 1   remainder 0   → digit 0
1 ÷ 16    = 0   remainder 1   → digit 1

Reading remainders bottom-to-top: 1064H
```

So the target address = `1064H` (= 4196 decimal).

Step 2 — Find PC (next instruction after the branch):

```
PC = 1000H + 4 = 1004H
Convert to decimal: 1×4096 + 0 + 0 + 4 = 4100
```

Step 3 — Apply the formula:

```
offset = target – PC = 4196 – 4100 = 96
```

Step 4 — Verify:

```
PC + offset = 4100 + 96 = 4196 = 1064H ✓ (matches our target address)
```

Answer: offset = +96 (a forward branch).

Key lesson from this question: Always be careful about *what the displacement is measured from*. Here, "100 bytes after the branch instruction" is NOT the same as "offset = 100" — because the offset used in the actual PC-relative calculation must be measured from the **PC** (next instruction), not from the branch instruction's own address. Since PC is already 4 bytes ahead of the branch instruction, the real offset (96) ends up being 4 less than the raw displacement (100) that was described in the problem.

Indexed Addressing

The core idea: **the effective address of the operand is calculated by adding an offset (given in the instruction) to the value stored in an *index register*.** This is conceptually very similar to relative addressing (PC + offset), except here the "base" being added to isn't the Program Counter — it's a dedicated **index register** that can be set to whatever value the programmer needs.

Breaking down the mechanism

- **Effective Address = Offset + Index Register**
- **Offset**: a constant value encoded directly in the instruction (this typically represents the *starting/base address* of a data structure, like an array).
- **Index register**: a register (either a special-purpose register built specifically for this role, or an ordinary general-purpose register) whose value represents *how far into that data structure* to go — i.e., which element to access.

Why this is more general than relative addressing

Relative addressing always adds the offset to the **PC** — a register the CPU manages automatically as it executes instructions. Indexed addressing instead adds the offset to a register **you** (the programmer/compiler) explicitly load with whatever value you want — this makes it far more flexible for things unrelated to program flow, like navigating through data structures.

Walking through the example

```
LOAD R1, 1050(R3) // R1 = Mem[1050 + R3]
```

- `LOAD` = opcode
- `R1` = destination register

- `1050` = the offset (encoded directly in the instruction) — this represents the **base/starting address of an array**
- `(R3)` = the index register — this holds the **element number** (index) within that array

Execution:

1. CPU reads the offset value `1050` directly from the instruction (no memory access needed for this part).
2. CPU reads the current value in register R3 (say R3 = 4).
3. CPU computes the effective address: `1050 + 4 = 1054`.
4. CPU accesses `Mem[1054]` — this is the actual operand — and loads it into R1.

Why this is perfect for arrays

Here's the key insight the slide is making:

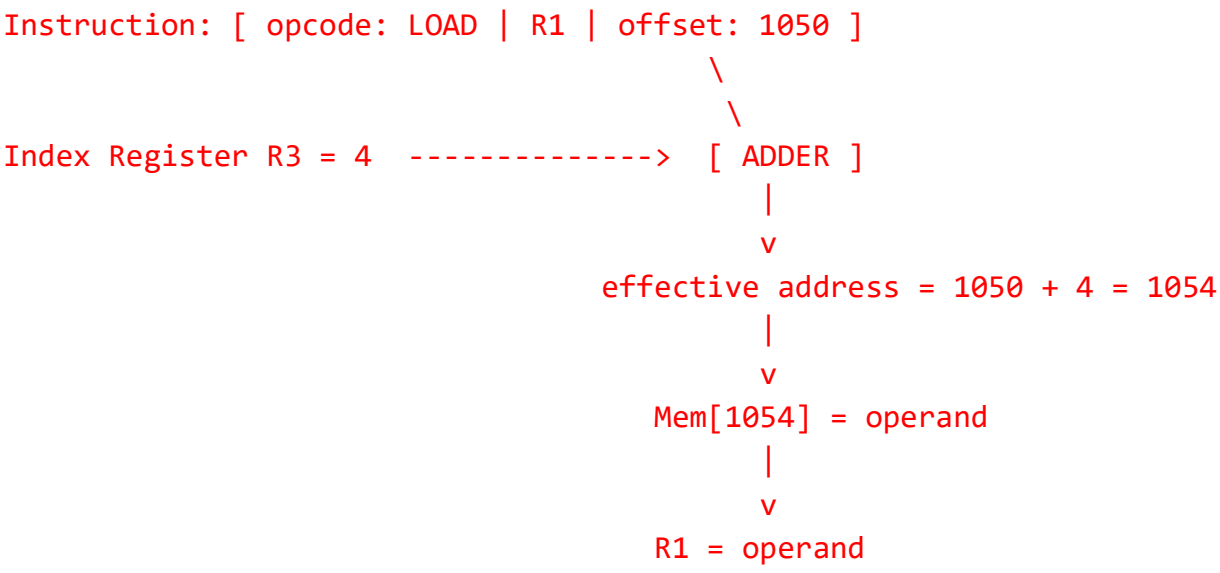
"Offset gives the starting address of the array, and the index register value specifies the array element to be used."

Think about how an array is laid out in memory: all its elements sit in *consecutive* memory locations, starting from some base address. If an array starts at address `1050`, then:

- Element 0 is at `1050 + 0`
- Element 1 is at `1050 + 1`
- Element 2 is at `1050 + 2`
- ... and so on

So if the **offset** in the instruction is fixed at `1050` (the array's starting address), and the **index register** holds a changing value (0, 1, 2, 3...), the *same instruction* `LOAD R1, 1050(R3)` can fetch a *different* array element just by changing what's in R3 beforehand. This is exactly how loops that scan through arrays work at the machine level — increment the index register each iteration, and reuse the same load instruction.

Box diagram



Comparing indexed addressing to relative addressing

Aspect	Relative (PC-relative) Addressing	Indexed Addressing
Register added to offset	PC (Program Counter — automatic)	Index register (explicitly set by the program)
Typical purpose	Branches, jumps, loops in control flow	Array/table element access
Who controls the register value	The CPU (via instruction fetch cycle)	The programmer/compiler (via explicit instructions like `ADD R3, R3, #1`)
Formula	Effective address = PC + offset	Effective address = Offset + Index Register

Questions

1. An array starts at memory address `2000H`. Using indexed addressing with instruction `LOAD R1, 2000(R4)`, if R4 = 6, calculate the effective address (in hex) and explain what element of the array is being accessed (assume each array element is 1 byte).

2. Explain, step by step, how the single instruction `LOAD R1, 2000(R4)` can be reused inside a loop to read *every* element of a 10-element array, without needing 10 different instructions. What must change between iterations?

3. A 2D array's row starts at address `3000H`, and each element in the row is 4 bytes wide (a word). Using `LOAD R2, 3000(R5)`, what value must be placed in R5 to access the 3rd element of the row (index 2, since indexing starts at 0)? Calculate the effective address.

4. Explain the conceptual difference between the "offset" in indexed addressing and the "offset" in relative addressing, even though both are added to a register to compute an effective address.

5. Given the instruction `LOAD R1, 5000(R3)`, if R3 currently holds `-3` (a negative index), is this a valid operation? Explain what this would mean in terms of accessing memory relative to the base address `5000H`, and calculate the effective address.

Model Answers (with explanations)

1. Array at `2000H`, `LOAD R1, 2000(R4)`, R4 = 6

Explanation: The offset (`2000H`) represents the array's base address, and R4 represents which element (index) to access. Since elements are 1 byte each, each increment of the index moves exactly 1 byte forward in memory — so we can add the values directly without extra scaling.

```
Convert 2000H to decimal: 2×4096 + 0 + 0 + 0 = 8192
Effective address = 8192 + 6 = 8198

Convert 8198 back to hex:
```

```
8198 ÷ 16 = 512 remainder 6  → digit 6
512 ÷ 16  = 32  remainder 0  → digit 0
32 ÷ 16   = 2   remainder 0  → digit 0
2 ÷ 16    = 0   remainder 2  → digit 2

Reading bottom to top: 2006H
```

Answer: Effective address = 2006H. This accesses **element index 6** of the array (the 7th element, since indexing starts at 0) — because R4 = 6 tells the CPU to move exactly 6 byte-positions past the array's starting address.

2. Reusing one instruction to loop through a 10-element array

Explanation: The whole point of indexed addressing is that the *offset* (base address, `2000`) stays fixed in the instruction — it never needs to change, since the array itself doesn't move in memory. What changes between iterations is only the **value inside the index register (R4)**, which is updated by a separate instruction between each use of the `LOAD`.

A loop would look like this:

```
MOV R4, #0            // R4 = 0  (start at first element)

loop:
  LOAD R1, 2000(R4)   // R1 = Mem[2000 + R4]  (fetch current element)
  ... process R1 ...
  ADD R4, R4, #1       // R4 = R4 + 1  (move to next element)
  CMP R4, #10
  BLT loop             // repeat if R4 < 10
```

Each time through the loop, the *same* `LOAD R1, 2000(R4)` instruction is executed, but because R4 has a different value each time (0, 1, 2, ... 9), it reads a *different* memory location each time: `2000, 2001, 2002, ... 2009`. This is far more efficient than writing 10 separate `LOAD` instructions with 10 different hardcoded addresses.

3. 2D array row at `3000H`, elements are 4 bytes wide, access index 2

Explanation: This is a slightly trickier variant. Unlike question 1 (1-byte elements, where the index value equals the byte offset directly), here each element takes up **4 bytes**. So to reach the 3rd element (index 2), the index register must hold not just "2," but the actual **byte offset** — which means R5 must be pre-scaled by the element size.

```
Byte offset needed = index × element size = 2 × 4 = 8

So R5 must be set to 8 (not 2), before executing LOAD R2, 3000(R5).
```

```
Convert 3000H to decimal: 3×4096 + 0 + 0 + 0 = 12288
Effective address = 12288 + 8 = 12296

Convert 12296 to hex:
12296 ÷ 16 = 768 remainder 8  → digit 8
768 ÷ 16   = 48  remainder 0  → digit 0
48 ÷ 16    = 3   remainder 0  → digit 0
3 ÷ 16     = 0   remainder 3  → digit 3

Reading bottom to top: 3008H
```

Answer: R5 must be set to 8 (the pre-scaled byte offset), giving effective address = 3008H. This demonstrates a common real-world detail: when array elements are larger than 1 byte, the compiler must multiply the logical index by the element size *before* placing it in the index register — otherwise the wrong memory location gets accessed.

4. Conceptual difference between "offset" in indexed vs. relative addressing

Explanation: Although both formulas look structurally identical (`base + offset`), what each "offset" *represents* is fundamentally different:

- In **relative addressing**, the offset represents a small **displacement in program flow** — "how many instructions away is the branch target." It's tied to the *code* itself and how far one instruction is from another.
- In **indexed addressing**, the offset represents the **starting address of a data structure** (like an array) — it's tied to *data*, not code. The thing that varies (the index register) represents "which data element," not "how far away in the instruction stream."

In short: relative addressing's offset is about *navigating code*, while indexed addressing's offset is about *navigating data*. This is also why relative addressing typically adds to the **PC** (a code-tracking register), while indexed addressing typically adds to a **general-purpose or index register** (a data-tracking register) that the programmer explicitly manages.

5. `LOAD R1, 5000(R3)`, R3 = -3

Explanation: Yes, this is a perfectly valid operation — index registers can legitimately hold negative values, since they're typically stored as signed numbers. A negative index simply means "access a memory location *before* the base address" rather than after it.

```
Convert 5000H to decimal: 5×4096 + 0 + 0 + 0 = 20480
Effective address = 20480 + (-3) = 20477

Convert 20477 to hex:
20477 ÷ 16 = 1279 remainder 13 → digit D (13 in hex = D)
1279 ÷ 16  = 79   remainder 15 → digit F
```

```
79 ÷ 16    = 4    remainder 15 → digit F
4 ÷ 16     = 0    remainder 4  → digit 4

Reading bottom to top: 4FFDH
```

Answer: Effective address = 4FFDH. This means the operand accessed is actually **3 bytes before** the array's stated base address (`5000H`) — a location technically "outside" the array's normal bounds (sometimes called accessing "before the array" or, in buggy code, an out-of-bounds/underflow access). This illustrates why programmers must be careful that index register values stay within the valid range for whatever data structure they're indexing into — a negative or overly large index can accidentally read unrelated memory.

The Key Difference: *Who* controls the register, and *why* it changes

Both relative addressing and indexed addressing use the same basic formula: `effective address = offset + [some register]`. The difference is entirely about **which register gets used**, and **who is in charge of what value goes into it**.

Relative Addressing: uses the PC (automatic, CPU-controlled)

The **PC (Program Counter)** is a special register that the CPU manages **entirely on its own**. You, as the programmer, never write an instruction like `MOV PC, #5000` in normal code. Instead:

- Every time the CPU fetches an instruction, it **automatically** increments the PC to point to the next instruction.
- This happens as a side-effect of just running the program — you don't control it directly.

So when relative addressing computes `PC + offset`, the PC value is **whatever it naturally is at that point in execution** — it's tied to *where the program currently is*, not something you can arbitrarily set to any value you like.

This is why relative addressing is only good for one thing: describing a location relative to "here" in the code — like "jump 20 instructions forward" or "branch back to the loop start." You can't really use it to say "go fetch array element number 7," because the PC has nothing to do with arrays — it only reflects instruction flow.

Indexed Addressing: uses an index register (manual, programmer-controlled)

An **index register** (like R3, R4, or a dedicated index register) is just an ordinary register that **you explicitly load with whatever value you want**, using a normal instruction like:

```
MOV R4, #6        // "I am choosing to put 6 into R4"
ADD R4, R4, #1    // "I am choosing to increase R4 by 1"
```

Nothing about the CPU's instruction-fetching automatically changes R4 — **you** decide what goes into it, and **you** decide when it changes. This means you can make it represent *anything you want* — not just "position in the program," but things like:

- "which element of this array am I looking at" (0, 1, 2, 3...)
- "how far into this data structure have I moved"

Side-by-Side Example

Relative addressing (PC-controlled):

```
BEQ -20     // "branch back 20 bytes from here" — this only makes sense
            // because PC tracks "where we currently are in the code"
```

You didn't set the PC to "20 bytes ago" — the PC just naturally holds the current execution position, and the CPU updates it as it runs.

Indexed addressing (you-controlled):

```
MOV R4, #0          // YOU decide: "start at array element 0"
LOAD R1, 2000(R4)   // fetch array[0]
ADD R4, R4, #1       // YOU decide: "now move to element 1"
LOAD R1, 2000(R4)   // fetch array[1]
```

Here, R4 has *nothing to do* with where the CPU currently is in the program. It's just a number *you* are choosing and updating, to represent "which array slot am I interested in right now."

One-Sentence Summary

- **Relative addressing** = "how far from **where the program currently is executing**" (PC is set automatically by the CPU as it runs).
- **Indexed addressing** = "how far from **some data's starting point**" (the index register is set manually by your own instructions, so it can represent whatever you need — like an array index — not just code position).

That's why indexed addressing is described as "more general" — the PC can only ever mean one thing (current code position), but an index register can be repurposed to mean *anything the programmer wants it to mean*.

Stack Addressing

The core idea: **the operand isn't specified by an address, a register, or an offset at all — it's *implicit*. The instruction just says "operate on whatever is currently on top of the stack."**

What is a stack, first?

A **stack** is a region of memory organized as a **Last-In-First-Out (LIFO)** structure — like a stack of plates. You can only add a plate to the top (**push**) or remove the top plate (**pop**). You can't grab a plate from the middle without first removing everything above it.

```
Stack (grows this direction)
 +--------+
 |   15   |  <-- TOP (most recently pushed)
 +--------+
 |   42   |
 +--------+
 |    7   |
 +--------+
 |  ...   |
 +--------+
```

Why it's called "implicit" addressing

In every previous addressing mode you've studied, the instruction had to *explicitly* say where the operand was — a register number, a memory address, an offset, etc. Stack addressing is different: **the location is never written in the instruction at all.** Instead, there's an unwritten rule: "the operand is always whatever's on top of the stack right now." The instruction `ADD` doesn't need to say *which* two values to add — everyone just agrees the two topmost stack values are the ones involved.

This is why the slide says it was **used in zero-address machines** — because if operands are always implicitly "on top of the stack," instructions don't need *any* address fields at all. Compare this to register addressing (needs a few bits for a register number) or direct addressing (needs a full address field) — stack addressing needs **zero** operand bits, since "top of stack" is understood by convention.

The Stack Pointer (SP)

Since the CPU needs to know exactly *where* in memory the "top of the stack" currently is, it keeps a special register called the **Stack Pointer (SP)**. This is very similar in spirit to the PC (Program Counter) — a dedicated register the CPU manages automatically, except SP tracks "where's the top of the stack" instead of "where's the current instruction."

- **PUSH**, **POP**, **CALL**, and **RET** instructions **automatically update SP** as a side effect — you never manually calculate or set SP yourself in normal use. This is analogous to how the PC auto-increments as instructions execute.

Walking through the examples

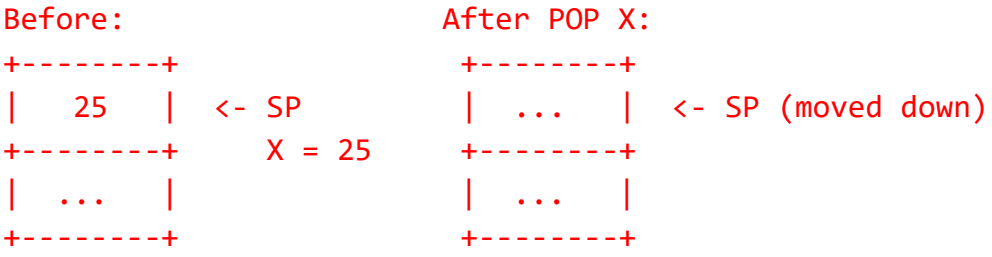
`PUSH X`

- Meaning: take the value of `X` and place it on top of the stack.
- Effect on memory: SP is adjusted (typically decremented, since most stacks grow "downward" in memory), and `X`'s value is written to the new top-of-stack location.

```
Before:                  After PUSH X (X = 25):
+--------+                +--------+
|  ...   |  <- SP         |   25   |  <- SP (new top)
+--------+                +--------+
                           |  ...   |
                           +--------+
```

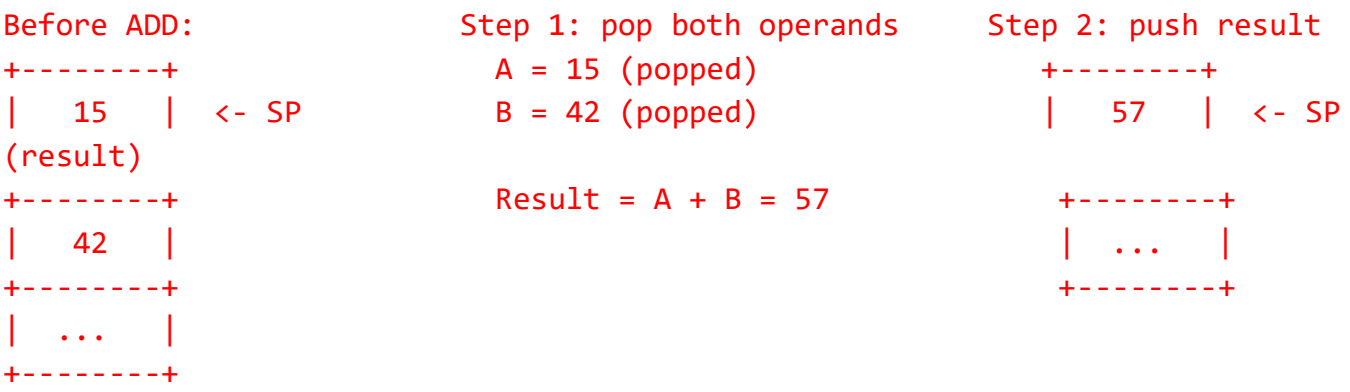
`POP X`

- Meaning: take the value currently on top of the stack, remove it from the stack, and store it into `X`.
- Effect: the value at the current top-of-stack location is read into `X`, and SP is adjusted (typically incremented) to point to the next item down.



`ADD` *(zero-operand form — this is the key example of "implicit" addressing)*

- Meaning: pop the top **two** values off the stack, add them together, and push the result back onto the stack.
- Notice: the instruction is just the word `ADD` — no operands specified anywhere! The CPU knows, by convention, to grab the two topmost stack values.



Why this connects to CALL/RET

`CALL` and `RET` (function calls and returns) also use the stack implicitly: `CALL` pushes the **return address** (essentially, the current PC) onto the stack before jumping to the subroutine, and `RET` pops that saved address back off the stack into the PC to resume execution where the call happened. This is why the slide groups `CALL`/`RET` together with `PUSH`/`POP` — all four instructions modify SP automatically, without the programmer ever touching SP directly.

Questions

1. Explain why stack addressing is described as requiring **zero operand bits** in the instruction. Compare this to how many bits a register addressing instruction would need for the same operation.

2. Trace the stack contents step by step for the following sequence, starting from an empty stack: `PUSH 10`, `PUSH 20`, `ADD`, `PUSH 5`, `ADD`. What is the final value on top of the stack?

3. Explain what happens to the Stack Pointer (SP) during a `PUSH` versus a `POP` instruction, and why they move in opposite directions.

4. Using a diagram, explain how `CALL` and `RET` use the stack to implement returning from a subroutine to the correct location in a program.

5. Explain why stack addressing is well-suited to evaluating arithmetic expressions (like calculator-style computation), and trace how the expression `(4 + 5) × 2` would be evaluated using `PUSH`/`ADD`/`MUL` instructions.

Model Answers (with explanations)

1. Why stack addressing needs zero operand bits

Explanation: In every other addressing mode, the instruction must specify *where* to find each operand — this costs bits. For example:

- Register addressing needs bits to identify *which* register (e.g., 4 bits for 16 registers, per operand).
- Direct addressing needs bits for a full memory address.

Stack addressing needs **none of this**, because the location of every operand is fixed by convention: "always the top of the stack" (and for two-operand instructions like `ADD`, "the top two items"). There's nothing to specify — the instruction can just be the opcode alone (e.g., `ADD`), with no address or register fields at all.

Comparison: A register-addressing `ADD R1, R2, R3` instruction needs roughly 3 × 4 = 12 bits just for the three register fields (on top of the opcode). A stack-addressing `ADD` needs **0** extra bits — just the opcode. This is exactly why it's called a "zero-address" instruction format.

2. Trace: `PUSH 10, PUSH 20, ADD, PUSH 5, ADD`

```
Start: empty stack []

PUSH 10:     Stack: [10]                      <- top
PUSH 20:     Stack: [10, 20]                  <- top (20 is now on top)
ADD:         pop 20, pop 10, compute 10+20=30, push 30
             Stack: [30]                      <- top
PUSH 5:      Stack: [30, 5]                   <- top (5 is now on top)
ADD:         pop 5, pop 30, compute 30+5=35, push 35
             Stack: [35]                      <- top
```

Final value on top of stack: 35

Explanation: Each `ADD` always operates on the **two most recently pushed** values, consuming (popping) both and replacing them with a single result. This is why the stack shrinks by one item net, every time `ADD` runs (2 items removed, 1 item added back).

3. SP behavior during PUSH vs. POP

Explanation:

- **PUSH**: adds a new item to the top of the stack. Since the stack is growing (getting one item taller), SP must move to point to this *new* top location — in most conventional (downward-growing) stacks, this means SP is **decremented** before the new value is written.
- **POP**: removes the current top item. Since the stack is shrinking, SP must move back to point to the *previous* item (the new top after removal) — this means SP is **incremented** after the value is read out.

Why opposite directions: PUSH and POP are inverse operations — PUSH grows the stack (moving SP toward "more items"), while POP shrinks it (moving SP toward "fewer items"). If the stack grows downward in memory (a very common convention), PUSH decreases SP's numeric address value, and POP increases it — they must move oppositely to correctly undo each other's effect.

4. How CALL/RET use the stack

Explanation:

```
Program flow:

Address 1000H: CALL 3000H      <- calling a subroutine at address 3000H
Address 1004H: (next instruction after the call)
...
Address 3000H: (subroutine code starts here)
...
Address 3050H: RET              <- return from subroutine
```

Step 1 — CALL executes:

```
1. CPU pushes the "return address" (1004H — the address right after the CALL)
onto the stack.
2. CPU jumps execution to 3000H (the subroutine).

Stack after CALL:
+---------+
| 1004H   |  <- SP (saved return address)
+---------+
```

Step 2 — subroutine runs, potentially pushing/popping its own local values, but the return address stays safely underneath any of its own stack usage.

Step 3 — RET executes:

```
1. CPU pops the value off the top of the stack — this retrieves 1004H.
2. CPU sets PC = 1004H, resuming execution right where the CALL left off.
```

```
Stack after RET:
+---------+
|  ...    |  <- SP (return address removed)
+---------+
```

Why this matters: This mechanism is exactly how function calls "remember" where to return to, even with nested calls — each `CALL` pushes its own return address, and each `RET` pops the most recent one, naturally handling nested/recursive function calls correctly because of the stack's LIFO ordering (the *most recently* called function is always the *first* to return).

5. Evaluating (4 + 5) × 2 using stack instructions

Explanation: Stack-based (zero-address) machines are naturally suited to arithmetic expressions because any expression can be converted into **postfix notation** (also called Reverse Polish Notation), where operators come *after* their operands. Postfix naturally matches how a stack works: push values as you encounter them, and when you hit an operator, pop the needed operands, compute, and push the result back — exactly like the `ADD` example above.

The expression `(4 + 5) × 2` in postfix is: `4 5 + 2 ×`

```
PUSH 4    Stack: [4]
PUSH 5    Stack: [4, 5]
ADD       pop 5, pop 4, compute 4+5=9, push 9
          Stack: [9]
PUSH 2    Stack: [9, 2]
MUL       pop 2, pop 9, compute 9×2=18, push 18
          Stack: [18]
```

Final result: 18 (matches `(4+5)×2 = 9×2 = 18` ✓)

Why this is elegant: Notice that no instruction ever needed to specify *which* values to operate on by name or address — the stack's natural LIFO order automatically ensures the right values are combined in the right order, as long as they're pushed in the correct sequence. This is precisely why early "zero-address" calculator-style machines (and even some modern virtual machines, like the JVM) use stack-based execution models — it elegantly handles nested expressions without needing any register or address bookkeeping.

Some Other Addressing Modes

1. Base Addressing

The core idea: the processor keeps a special-purpose register — the **base register** (also called a **segment register**) — and *every* memory address the program generates gets added to this base register before actually being used to access memory.

How it works:

- All addresses used within a program (whether from direct addressing, indexed addressing, etc.) are treated as *relative* offsets within a designated block/segment of memory.
- The **base register** holds the starting address of that block.
- **Final memory address = generated address + base register value**

```
Instruction generates address: 0500H
Base register holds:           2000H

Final effective address = 0500H + 2000H = 2500H
```

Why "allows easy movement of code and data in memory":

This is the key practical benefit. Suppose a program is written assuming it will be loaded starting at address `0000H`, using addresses like `0500H`, `0800H`, etc. throughout its instructions. If the operating system needs to load this program somewhere *else* in memory (say, starting at `2000H`, because `0000H` is already occupied by something else), the program's internal addresses don't need to be rewritten at all.

Instead, the OS simply sets the **base register** to `2000H`. Now every address the program generates automatically gets `2000H` added to it before hitting real memory — so `0500H` in the program's own "view" of memory becomes `2500H` in actual physical memory, without a single instruction needing to change.

This is essentially how memory **relocation** and **segmentation** work in operating systems — programs can be loaded anywhere in physical memory, and moved around later (e.g., during multitasking, when the OS swaps programs in and out of memory), just by updating the base register, rather than modifying every address in the program itself.

Mental model: Think of the base register as a building's floor number, and the program's own addresses as room numbers *within* that floor. "Room 5" always means something different depending on which floor you're on — you don't need to relabel every room if the whole floor's location within the building changes; you just update which floor you're referring to.

2. Autoincrement and Autodecrement Addressing

The core idea: this is built on top of **register indirect addressing** (where a register holds the address of the operand) — but with an added twist: **after** the operand is accessed, the register's value is *automatically* adjusted (incremented or decremented) by the CPU, without needing a separate instruction to do so.

Where it came from: the slide notes this was **first introduced in the PDP-11**, a highly influential minicomputer architecture from the 1970s — many of its addressing mode ideas went on to influence later processor designs.

How it works — Autoincrement:

```
Register R1 holds address 3000H
Instruction: LOAD R2, (R1)+     // access Mem[R1], THEN increment R1

Step 1: R2 = Mem[3000H]    (fetch operand)
Step 2: R1 = R1 + 1 = 3001H (automatic increment, after the access)
```

How it works — Autodecrement:

```
Register R1 holds address 3000H
Instruction: LOAD R2, -(R1)     // decrement R1 FIRST, then access Mem[R1]

Step 1: R1 = R1 - 1 = 2FFFH (automatic decrement, before the access)
Step 2: R2 = Mem[2FFFH]     (fetch operand)
```

(Note: whether the increment/decrement happens before or after the memory access depends on the specific mode — "autoincrement" typically adjusts after use, "autodecrement" typically adjusts before use, which is exactly what makes them convenient for stepping through memory forward or backward, respectively.)

Why this connects to `a++` and `a--` in C:

The slide's comparison to C's `a++` and `a--` operators is very apt. In C:

- `a++` (post-increment) uses the current value of `a` first, *then* increases it.
- `--a` or `a--` similarly ties incrementing/decrementing directly to the act of using the variable.

Autoincrement/autodecrement addressing does the exact same thing at the hardware level, but for a register that holds a *memory address* rather than an ordinary variable: using the address (to fetch data) and updating it happen together, as one combined action, rather than needing two separate instructions.

Why this is useful — array/string traversal without extra instructions:

Compare this to what you'd need *without* autoincrement, using plain register indirect addressing:

```
LOAD R2, (R1)      // fetch element
ADD  R1, R1, #1    // manually move to next element – a SEPARATE instruction
```

With autoincrement, both of these actions collapse into **one** instruction:

```
LOAD R2, (R1)+     // fetch element AND automatically advance R1
```

This saves an instruction on every single step of a loop that walks through memory (like scanning a string character-by-character, or processing array elements one at a time) — which, across a long loop, adds up to a meaningful reduction in the number of instructions executed.

Mental model: Think of it like an automatic page-turner while reading a book — instead of you having to read a page *and then* separately reach over and turn the page yourself (two actions), the autoincrement mechanism does both in one motion: read this page, and it's already ready to show you the next one.

compare all of those addressing methods immediate, direct,indirect, register, register indirect, relative, index , and stack

Orchestrated comprehensive comparison across eight addressing modes
Orchestrated comprehensive comparison across eight addressing modes

Comprehensive Comparison — All Addressing Modes

Addressing Mode	Where Operand Actually Is	What Instruction Contains	Memory Accesses Needed	Speed	Address Space	Typical Use Case	Example
Immediate	Inside the instruction itself	The actual data value	0	Fastest	N/A (not an address)	Constants, loop initializers	`ADDI #25`
Direct	In memory, at a fixed address	The memory address of the operand	1	Fast	Limited by address field width	Accessing known/fixed variables	`ADD R1, 20A6H`
Indirect	In memory, at an address stored in another memory location	Address of a location holding another address	2	Slowest (of the memory-based ones)	Large (not limited by instruction's address field)	Pointer-to-pointer scenarios	`ADD R1, (20A6H)`
Register	In a CPU register	The register number	0	Fastest (tied with immediate)	N/A — limited to number of registers	Fast temporary computation, RISC/load-store architectures	`ADD R1, R2, R3`
Register Indirect	In memory, at an address held in a register	The register number (which holds an address)	1	Fast	Large (limited only by register width)	Pointer dereferencing, array/linked-list traversal	`ADD R1, (R5)`
Relative (PC-Relative)	In memory, at PC + offset	A signed offset	1	Fast	Limited range (near current PC) but position-independent	Branches, loops, position-independent code	`BEQ -20`
Indexed	In memory, at offset + index register	A base offset (index register set separately)	1	Fast	Large (index register can hold large values)	Array/table traversal	`LOAD R1, 1050(R3)`

Stack	On top of the stack, at address held in SP	Nothing (fully implicit)	1 (via SP)	Fast	N/A — tied to stack region	Expression evaluation, function calls (CALL/RET)	`ADD` (no operands)

Grouped by Key Dimensions

By number of memory accesses (speed hierarchy)

```
0 accesses:  Immediate, Register          <- fastest, no memory touched
1 access:    Direct, Register Indirect,
             Relative, Indexed, Stack      <- one trip to memory
2 accesses:  Indirect                      <- slowest, two trips to memory
```

By what's actually stored in the instruction

```
Data itself:           Immediate
An address:             Direct, Relative, Indexed (offset), Stack (implicit —
no field at all)
A register number:      Register, Register Indirect
```

By address-space limitation

```
Limited by instruction's own bit-width:   Direct
Not limited (uses full register/memory width): Indirect, Register Indirect,
Indexed
Limited but position-independent:          Relative
```

The Core Pattern Across All Modes

Nearly every mode you've studied is really a variation on one question: **"How many hops does the CPU take to get from the instruction to the actual data?"**

```
0 hops:  Immediate (data IS in the instruction)
         Register (data IS in a named register)

1 hop:   Direct (instruction → memory)
         Register Indirect (instruction → register → memory)
         Relative (instruction → PC+offset → memory)
         Indexed (instruction → index register+offset → memory)
         Stack (instruction → SP → memory, implicitly)

2 hops:  Indirect (instruction → memory → memory)
```

The general trade-off across all modes

	Fewer hops (Immediate, Register)	More hops (Indirect)
Speed	Faster	Slower
Flexibility	Less flexible (fixed at compile time)	More flexible (can point anywhere, dynamically)
Instruction size	Smaller/compact	Similarly compact — the "cost" shows up in execution time, not instruction size

Special-purpose registers used by different modes

```
PC (Program Counter)    → used by Relative addressing
SP (Stack Pointer)      → used by Stack addressing
Base/Segment register   → used by Base addressing
General-purpose register → used by Register, Register Indirect, Indexed,
Autoincrement/decrement
```

This is really the unifying thread across the whole topic: **every addressing mode is just a different strategy for balancing speed, flexibility, and instruction size when telling the CPU "where's the data?"** — from "it's right here" (immediate) all the way to "follow this chain of pointers" (indirect), with everything else falling somewhere in between depending on what the program actually needs to do.